SubTropica

v1.1.0

Mathieu Giroux,[1,2] **Sebastian Mizera,**[1] **Giulio Salvatori**[2,3]

[1] *Department of Physics, Columbia University,*
538 West 120th Street, New York, NY 10027, USA

[2] *Institute for Advanced Study*
Einstein Drive, Princeton, NJ 08540, USA

[3] *Max-Planck-Institut für Physik, Werner-Heisenberg-Institut,*
D–85748 Garching bei München, Germany

giroux@ias.edu, sebastian.mizera@columbia.edu, salvatori@ias.edu

Abstract

We present SubTropica, a Mathematica package that performs symbolic integration of multi-polylogarithmic integrals using recent advances in tropical geometry. It focuses on the class of linearly-reducible *Euler integrals*, such as Feynman integrals, and expands them using a tropical subtraction scheme. The engine behind it is HyperIntica, a native Mathematica package for hyperlogarithm integration that can be used independently.

This paper documents both packages and illustrates their usage on examples from across different physics applications. Additionally, we introduce an AI-driven library of Feynman integrals, which catalogs diagrams discussed in the literature and serves as a database for computed results. Its online version is available at

subtropi.ca

and features a graphical user interface for diagram input and retrieval of records.

Contents

1 Introduction **2**

2 Manual **6**
- 2.1 Installation . . . 6
- 2.2 Computing Feynman integrals in a hurry . . . 7
- 2.3 Architecture and function overview . . . 8
- 2.4 Numerical evaluation and verification . . . 12
- 2.5 `SubTropica` library . . . 13

3 Theory and algorithms **14**
- 3.1 An invitation to Euler integrals . . . 14
- 3.2 Tropical algorithms . . . 17
  - 3.2.1 Divergences . . . 17
  - 3.2.2 Locally finite expansion . . . 21
  - 3.2.3 A worked out example . . . 26
- 3.3 Integration of locally finite integrands . . . 28
  - 3.3.1 Hyperlogarithms and their derivatives . . . 28
  - 3.3.2 The integration strategy . . . 29
  - 3.3.3 Regularization at the endpoints . . . 31
  - 3.3.4 A worked example . . . 33
- 3.4 Integration order and linear reducibility . . . 34
  - 3.4.1 Fubini reduction . . . 35
  - 3.4.2 Ordering orders . . . 35

4 Detailed examples **37**
- 4.1 Scalar Feynman integrals . . . 37
  - 4.1.1 A one-loop box with algebraic letters (Fig. 5 (left)) . . . 37
  - 4.1.2 A two-loop triangle-box integral (Fig. 5 (right)). . . . 38
  - 4.1.3 A cutting-edge example (Fig. 1) . . . 39
- 4.2 Linearized propagators and tensor numerators . . . 40
- 4.3 Advanced examples: Beyond Feynman integrals . . . 41
  - 4.3.1 Gravitational energy-energy correlators . . . 41
  - 4.3.2 Fourier transforms in small-x physics . . . 44
- 4.4 A standalone `HyperIntica` example . . . 47

5 Conclusion **49**

A Function reference **52**

A.1 Detailed package configuration: `ConfigureSubTropica` 52
A.2 Complete option reference: `STIntegrate` 53
A.3 Numerical evaluation: `STNIntegrate` 62
A.4 Verification: `STVerify` 63
A.5 Integration functions: `HyperIntica` 64
A.6 Tropical subtraction utilities 69
A.6.1 Tropical pre-analysis: `STPreAnalysis` 70
A.6.2 Nilsson–Passare continuation: `STTropicalContinuation` . . 70
A.6.3 Tropical subtraction: `STExpandIntegral` 71
A.6.4 Scaling analysis: `STFactor` 72
A.6.5 Series mapping: `STMapSeries` 72
A.7 Linear reducibility: `STFasterFubini` 72

B Notation used in `HyperIntica` **73**

1 Introduction

A recurring bottleneck in perturbative calculations across high-energy physics is the evaluation of multi-dimensional integrals. Examples include Feynman integrals, phase-space integrals, energy and cosmological correlators, and so on. They belong to a common class known as *Euler integrals*:

$$I(\varepsilon, \mathbf{s}) = \int_0^\infty \mathrm{d}x_1 \cdots \mathrm{d}x_n \prod_i P_i(x, \mathbf{s})^{a_i + b_i \varepsilon}\,, \tag{1.1}$$

where $P_i(x, \mathbf{s})$ are polynomials in the integration variables x with coefficients collectively denoted by $\mathbf{s}$, the exponents a_i, b_i are fixed constants, and $\varepsilon > 0$ is a small real parameter (typically the dimensional regulator $\varepsilon = (4 - \mathrm{D})/2$ in the Feynman integral context).

In this paper, we introduce `SubTropica`, a MATHEMATICA package that evaluates integrals of the form (1.1) as Laurent expansions in ε,

$$I(\varepsilon, \mathbf{s}) = \sum_{i=-N}^{\infty} \varepsilon^i \, I_i(\mathbf{s})\,, \tag{1.2}$$

expressing the coefficients $I_i(\mathbf{s})$ in terms of *hyperlogarithms*. The central difficulty is that, often, the integrand of (1.1) cannot simply be expanded in ε and integrated term by term, which would give meaningless expressions. To overcome this, `Sub-`

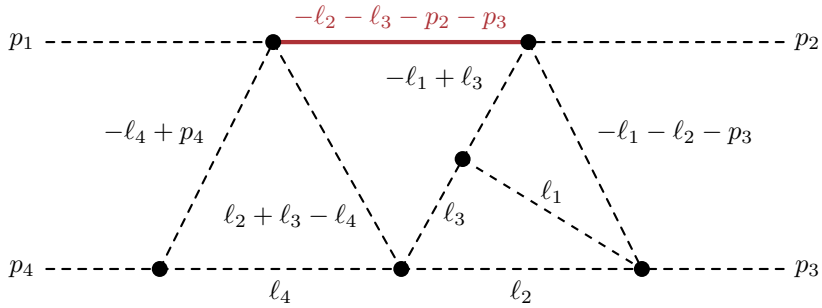

Figure 1: A four-loop Feynman diagram with nine propagators, one massive internal (red, solid) edge ($m^2 = 1$), and four massless (black, dashed) external legs. `SubTropica` evaluates for the first time the corresponding integral through $\mathcal{O}(\varepsilon^0)$. See Sec. 4.1.3 for details.

`Tropica` implements the tropical subtraction method of [1] (see also [2, 3]), which uses the geometry of the Newton polytope associated with the integrand to systematically construct counter-terms that render each integral locally finite. The resulting *locally finite* integrands can then be expanded in ε at the integrand level and integrated one variable at a time using, e.g., the hyperlogarithm integration algorithms of Brown and Panzer [4, 5]. This last step requires the integral to be *linearly reducible*, meaning that at each stage of the iterated integration, all singularities remain linear in the current integration variable. While not all Euler integrals satisfy this condition, a large and practically important class does, including, e.g., many multi-loop Feynman integrals relevant for phenomenology.

As a demonstration of its capabilities, `SubTropica` can tackle cutting-edge examples such as the four-loop, nine-propagator massive diagram shown in Fig. 1.

Relation to prior work. While powerful implementations of the hyperlogarithm integration algorithm exist in Maple (`HyperlogProcedures` [6] and `HyperInt` [7]) and FORM (`HyperFORM` [8], `Forcer` [9]), a significant part of the high-energy physics community relies on Mathematica as its primary computational environment, where tools for amplitude generation and symbolic manipulation [10–15], integration-by-parts reduction [16–24], differential equations and canonical forms [25–30], numerical evaluation [31–34], evaluation of special functions and iterated integrals [35–37], symbol and singularity analysis [38–42], parametric and GKZ methods [43], spinor-helicity techniques [44–47], and tensor reduction [48] are readily available or interoperable. To keep the entire workflow within a single platform, `SubTropica` ships with `HyperIntica`, a self-contained

MATHEMATICA reimplementation of the `HyperInt` algorithm that can also be used as a standalone package for evaluating iterated integrals of hyperlogarithms.

Repository. The up-to-date version of `SubTropica` is hosted open source on GITHUB:

`github.com/SubTropica/SubTropica`

which also contains the associated examples and documentation. The code is released under the MIT license.

Library. Together with `SubTropica`, we publish an extensive library of diagrams that have been computed in the literature. Each entry contains detailed records about the diagram itself, papers in which it appeared, classes of functions it evaluates to, etc. When available, it also has the result of computation with `SubTropica` in terms of hyperlogarithms. We invite all readers to submit new entries to the library. The library can be accessed through the graphical user interface (GUI) provided in `SubTropica`, as well as online at

`subtropi.ca`

The online version allows one to draw, browse, and download diagrams, but the integration features require the MATHEMATICA backend. The library is released under the CC BY-NC-SA 4.0 license, including restrictions on machine learning use.

Outline. The paper is organized as follows. Sec. 2 serves as a quick-start user manual: installation, basic functionality, and an overview of the package architecture. In Sec. 3, we review the theoretical ingredients: Euler integrals and their Newton polytopes, the tropical subtraction algorithm, and the hyperlogarithm integration strategy, including a fully worked example. In Sec. 4, we present detailed examples ranging from Feynman integrals to applications beyond the scattering-amplitude context. We conclude in Sec. 5 with a summary and outlook. Technical details on the `STIntegrate` and `ConfigureSubTropica` interfaces, as well as `HyperIntica`'s conventions, are collected in the appendices. To ensure reproducibility, all code snippets quoted in this paper are collected and checked in the accompanying notebook `PaperChecks.wl` on GITHUB.

AI use. The development of `SubTropica` was assisted by Claude Opus 4.6–7 at all stages: from algebraic manipulations (cross-checking computations, find-

ing rationalization of square roots, statistical analysis), through coding and algorithms (optimization, benchmarking, debugging, designing the user interface), to the library construction (retrieval and parsing of records, numerical verification, matching system).

Feynman integral conventions. Throughout this paper and the accompanying package, we adopt the following normalization convention for Feynman integrals:

$$I_{\vec{\nu}}(\mathbf{s}) \equiv \frac{\mathrm{e}^{\varepsilon L \gamma_{\mathrm{E}}}}{(i\pi^{\mathrm{D}/2})^L} \int \frac{\mathcal{N}\, \mathrm{d}^{\mathrm{D}}\ell_1\, \mathrm{d}^{\mathrm{D}}\ell_2 \cdots \mathrm{d}^{\mathrm{D}}\ell_L}{(q_1^2 - m_1^2)^{\nu_1}(q_2^2 - m_2^2)^{\nu_2} \cdots (q_N^2 - m_N^2)^{\nu_N}}\,, \tag{1.3}$$

where $\{\ell_a\}_{a=1}^L$ represents the set of the L independent loop momenta, and q_e and m_e are the momenta and masses associated with the internal edges. The exponents ν_i are the integer powers of the respective propagators or irreducible scalar products, while $\mathcal{N}$ is a numerator polynomial in the loop momenta. The vector $\mathbf{s}$ collectively refers to the kinematic variables, such as masses and Mandelstam invariants. The quantity γ_{E} is a shorthand for the Euler constant $-\Gamma'(1) \approx 0.577$. The Feynman $i\epsilon$ prescription is left implicit. We use mostly-minus $(+ - \ldots -)$ conventions.

The resulting Schwinger-parameter representation is

$$I_{\vec{\nu}}(\mathbf{s}) = \Gamma(\omega) \left(\prod_{e=1}^{E} \frac{(-1)^{\nu_e}}{\Gamma(\nu_e)} \right) \int \frac{\mathrm{d}^E x}{\mathrm{GL}(1)} \prod_{e=1}^{E} x_e^{\nu_e - 1} \frac{\tilde{\mathcal{N}}(\mathbf{s})}{\mathcal{U}^{\mathrm{D}/2-\omega}[-\mathcal{F}(\mathbf{s})]^{\omega}}\,, \tag{1.4}$$

where E denotes the number of internal edges (those with $\nu_e > 0$), x_e denotes the Schwinger parameter associated with every internal edge e, and

$$\omega \equiv \sum_{e=1}^{E} \nu_e - \frac{L\mathrm{D}}{2}\,, \tag{1.5}$$

is the superficial degree of divergence, see, e.g., [49, App. A]. The factors $\mathcal{U}$ and $\mathcal{F}$ are the first and second Symanzik polynomials respectively, while $\tilde{\mathcal{N}}$ is the numerator polynomial. Modding out by the gauge redundancy GL(1) can be done by fixing one of the Schwinger parameters, say $x_E = 1$.

Measure conventions. In the text, we write x_i with the standard measure $\prod_i \mathrm{d}x_i$ (as in (1.4)), and α_i with the logarithmic measure $\prod_i \mathrm{d}\log\alpha_i$ (as in (3.1)). The variables are the same ($x_i = \alpha_i$) and the different labels signal which measure is in play. The $\mathrm{d}\log$ form is natural when tropicalizing integrands. The $\mathrm{d}x$ form is standard in the Feynman-integral literature and is used by the integrator

HyperIntica. The two measures simply differ by the Jacobian $\prod_i x_i$.

2 Manual

This section explains how to install SubTropica and its dependencies, outlines its main functions, and provides the complete documentation.

2.1 Installation

SubTropica can be installed by running

```
PacletInstall["https://subtropi.ca/SubTropica.paclet"]
```

directly in MATHEMATICA. Alternatively, it is available at the GITHUB repository [50] and can be downloaded or cloned directly. The package then needs to be moved to MATHEMATICA's $Path directory manually. SubTropica requires MATHEMATICA 12.1 or later.

Once the package is installed, it can be loaded and configured from anywhere with[1]

```
Get["SubTropica`"];
```

It can be configured by typing

```
ConfigureSubTropica[(* required *)
                    PolymakePath   -> "path/to/polymake",
                    (* recommended *)
                    FiniteFlowPath -> "path/to/FiniteFlow",
                    SPQRPath       -> "path/to/SPQR",
                    PythonPath     -> "path/to/python",
                    (* optional *)
                    GinshPath      -> "path/to/ginsh",
                    MaplePath      -> "path/to/maple",
                    FIESTAPath     -> "path/to/FIESTA"
                    ];
```

[1] All paths set via ConfigureSubTropica are saved to a persistent configuration file ($UserBaseDirectory/Kernel/SubTropicaConfig.m) and automatically restored on every subsequent Get["SubTropica`"], so ConfigureSubTropica needs to be called only once, unless the configuration needs updating. The saved configuration can be cleared with STResetConfig[].

The only required dependency is `polymake` [51, 52].

All other dependencies are optional. A working installation of PYTHON is highly recommended, as it is needed for the graphical user interface (GUI) and for `pySecDec` [31]. (The latter should be installed via `pip` into the PYTHON interpreter set as `PythonPath`.) The package `ginsh` (the `GiNaC` interactive shell [53]) is needed only to evaluate hyperlogarithms numerically, and MAPLE can be useful for cross-checks, since `SubTropica` includes `HyperIntica`, a native MATHEMATICA reimplementation of `HyperInt` [7]. Finally, `pySecDec` and `FIESTA` [32] can be used to validate the computed results numerically.

For complicated examples, we recommend the user to work with the libraries `FiniteFlow` [24] and `SPQR` [54] to enable finite-field arithmetic in the partial-fraction step of the integration algorithm within `HyperIntica`. When the function `Get["SubTropica`"]` is called, the package silently checks whether they are already available on MATHEMATICA's `$Path` and loads them automatically if so. In that case, `FiniteFlowPath` and `SPQRPath` need not be set. If either library is loaded, `PartialFractions` automatically switches from MATHEMATICA's built-in `PolynomialQuotient` to finite-field reconstruction via `SPQRPolynomialQuotient`, which avoids intermediate expression swell on complicated integrands.

Tip. On macOS, the easiest installation route for most packages mentioned above is HOMEBREW [55]. For example, the default path for `polymake` is `/opt/homebrew/bin/polymake`. After installation, we recommend running

```
STBenchmark[]
```

which verifies the dependencies are installed correctly and a suite of test examples passes all the checks (`"Suite" -> "Short"` or `"Long"` selects the length of the benchmark).

2.2 Computing Feynman integrals in a hurry

The entry point of `SubTropica` is the function `STIntegrate`. While a rich set of calling conventions is available for scripted usage (see Sec. 2.3 and App. A), the simplest way to get started is to call

```
STIntegrate[]
```

with no arguments. This command opens a GUI that covers the entire workflow—from drawing the diagram to obtaining the integrated result—without writing any

code. The GUI features an interactive tutorial after launching it for the first time. After a diagram is drawn and all the necessary options (space-time dimension, requested ε order) are specified, one simply clicks the "Integrate" button to start integrating.

2.3 Architecture and function overview

Putting aside the GUI, internally, `SubTropica` consists of three distinct modules. The first implements the tropical subtraction algorithm discussed in Sec. 3.2, whose main functions are described further in App. A.6. The second, called `HyperIntica`, implements the integration of hyperlogarithms [7], reviewed in Sec. 3.3. A third module provides a Feynman graph interface and combines the previous modules through the single top-level function `STIntegrate`, which automates GL(1) gauge fixing for projective integrals, tropical continuation and subtraction, linear reducibility analysis, and (parallel) integration via `HyperIntica`.

While the GUI presented above is tailored to Feynman integrals (Form 0), `STIntegrate` also accepts scripted input in several formats: a Feynman graph specification (Form 1), a propagator list with support for masses, numerators, and linearized propagators (Form 2), or a direct Euler integrand for integrals not tied to a Feynman diagram (Form 3). See the summarizing paragraph below for the full list of calling conventions.

This variety of allowed inputs for `STIntegrate` makes the full tropical subtraction and integration pipeline available to generic Euler integrals, as illustrated in the examples of Sec. 4. The main lower-level functions listed in Tab. 1 can also be called individually for finer control over each step. The complete documentation of `STIntegrate` (input formats, option reference, checkpointing, and face targeting) is given in App. A.2. See Fig. 6 for a graphical summary of its internal workflow.

Summary of argument conventions for `STIntegrate`

Form 0. `STIntegrate[]`. Calling `STIntegrate[]` with no arguments opens the interactive Feynman diagram interface above, where the user can draw a diagram, assign masses, configure options, and launch the integration from the GUI interactively. In the GUI text fields, the suggested syntax is to use underscore notation for subscripts (e.g., `m_1` for m_1, `p_1` for p_1, `l_1` for ℓ_1). Then, masses are automatically squared/homogenized in the output ($m_1 \to$ `mm1`, $M_1 \to$ `MM1`). Strings such as `m1` are treated as literal symbols and will not be automatically homogenized, with the exception of `m` and `M` which are recognized as mass parameters. ✧

Function	**Description**	**Ref.**
Tropical subtractions		
`STPreAnalysis`	Identify divergent rays and faces of the Newton polytope	App. A.6.1
`STTropicalContinuation`	Perform Nilsson–Passare analytical continuation along specified rays	App. A.6.2
`STExpandIntegral`	Expand an Euler integrand into locally finite counter-terms via tropical subtraction	App. A.6.3
`STFactor`	Expose $(\cdots)^{\varepsilon}$ scaling behavior along rays	App. A.6.4
`STFasterFubini`	Find a linearly reducible integration order	App. A.7
Integration engine (`HyperIntica`)		
`HyperIntica`	Integrate a hyperlogarithm-valued integrand in specified variables	App. A.5
`HyperD`	Differentiate a hyperlogarithm expression with respect to a parameter	App. A.5
`HyperSeries`	Expand a hyperlogarithm expression as a series around a point	App. A.5
`FibrationBasis`	Compute the fibration basis of a hyperlogarithm	App. A.5
`ConvertToSymbol`	Convert a hyperlogarithm expression to symbol notation	App. A.5
Numerical evaluation and checks		
`STNIntegrate`	Evaluate an Euler integral numerically via `pySecDec`, `FIESTA`, `AMFlow` or `feyntrop`	Sec. 2.4
`STVerify`	Cross-check a symbolic result against numerical sector decomposition	Sec. 2.4
`STToGinsh`	Evaluate hyperlogarithms numerically via `ginsh`	App. A.5

Table 1: Main lower-level functions in `SubTropica` and `HyperIntica`.

Form 1. `STIntegrate[{edges, nodes}, opts]`. The input format follows that of SOFΣA [40]. A diagram is specified by two lists, `edges` and `nodes`. Each entry of `edges` is a tuple `{{i, j}, m}`, meaning that vertices `i` and `j` are connected by a propagator with mass `m` (order irrelevant). Each entry of `nodes` is a tuple `{i, M}`, meaning an external momentum p_i with $p_i^2 = \mathtt{M}^2$ is attached to vertex `i`. Here, the order *does* matter, as momenta are labeled $p_1, p_2, \ldots$ by their position in the list.

As an example, a two-mass box corresponds to

```
boxEdges = {{{1, 2}, m1}, {{2, 3}, 0}, {{3, 4}, 0}, {{1, 4}, 0}};
boxNodes = {{1, m1}, {2, 0}, {3, 0}, {4, M4}};
STIntegrate[{boxEdges, boxNodes}]
(* Out: Log[MM4/s12]-Log[1-s23/mm1]/(eps(mm1(MM4-s12)+s12 s23)) + ... *)
```

We observe that the code automatically assigns independent Mandelstam invariants of the form `sij...` $= (p_i + p_j + \cdots)^2$ in a cyclic basis. For example, for six external particles the invariants are `s12`, `s23`, `s34`, `s45`, `s56`, `s16`, `s123`, `s234`, and `s345` (in D dimensions). We also see that `SubTropica` automatically squares external (and internal) masses $\mathtt{M}_i^2$ ($\mathtt{m}_j^2$) into the symbols `MMi` (`mmi`) and uses $\varepsilon =$ `eps` as the default regulator symbol.

Note that the diagram can also be drawn interactively via the SOFΣA interface

```
{edges, nodes} = FeynmanDraw
```

which returns `{edges, nodes}` with generic masses; non-generic mass configurations can then be imposed via usual substitution rules "`//.{...}`" on `{edges, nodes}`. The diagram can be visualized with `FeynmanPlot[{edges, nodes}]`. The option `"LabelInternalEdges" -> True` additionally numbers the internal edges in the order they were drawn. ✧

Form 2. `STIntegrate[{propagators}, opts]`. The function also accepts a list of propagators and numerators

```
STIntegrate[{propagator1, propagator2, ..., numerator1, ...},
            "Exponents" -> {1, 1, ..., -1, ...}]
```

This representation conveniently allows the inclusion of a user-defined numerator depending on the loop momenta. Numerators and (integer powers of) inverse propagators are differentiated by the code using the option `"Exponents"`, which will be discussed more below. Similarly, vector numerators can be passed using (arbitrary) reference vectors such as q^μ and q^ν. An example illustrating the syntax

on the "squared tadpole" integral

$$q^\mu q^\nu \int \frac{\mathrm{d}^{\mathrm{D}}\ell_1}{i\pi^{\mathrm{D}/2}} \frac{\ell_1^\mu \ell_1^\nu}{[\ell_1^2 - m^2]^2} \qquad (\text{with } \mathrm{D} = 4 - 2\varepsilon)\,, \tag{2.1}$$

would be

```
STIntegrate[{ℓ[1]^2 - mm, q[1]·ℓ[1], q[2]·ℓ[1]}, "Exponents" -> {2, -1, -1}]
(*Out (assuming mm>0): mm q[1]·q[2]/2 (1/eps - (Log[mm] - 1)) + O[eps] *)
```

where the exponent 2 indicates a squared propagator and exponents -1 indicate linear numerators. Note that when tensor numerators are present, Lorentz dot products (see also Sec. 4.2) must be entered using MATHEMATICA's

```
a \[CenterDot] b        (* or *)        CenterDot[a,b]
```

operator or, equivalently, using the dot " `a.b` " symbol on the keyboard. Note that the regular multiplication " `a*b` " is allowed but not recommended since it might lead to contraction ambiguities. ✧

Form 3. `STIntegrate[integrand, {x,0,1}, {y,0,∞}, . . . , opts]`. In this case `STIntegrate` takes an integrand in MATHEMATICA-style integration limits. Note that bare symbol inputs (such as `STIntegrate[integrand, x, y, . . . ]`) default to $[0, \infty)$.

For generic Euler integrals that are, e.g., not tied to a Feynman diagram, `STIntegrate` accepts a syntax modeled on MATHEMATICA's built-in `Integrate`:

```
STIntegrate[integrand, {x, 0, 1}, {y, 0, ∞}, opts]
```

Each `{var, lo, hi}` triple specifies an integration variable and its bounds. The allowed bounds are 0, 1, and ∞. When limits are omitted, bare symbols default to $[0, \infty)$:

```
STIntegrate[integrand, x, y, opts] (* same as {x,0,∞}, {y,0,∞} *)
```

Before moving on, let us note that `STIntegrate` also accepts a pre-built tuple generated by internal `SubTropica` functions

```
STIntegrate[{prefactor, integrand, variables, coefficients}]
```

where `prefactor` is a multiplicative factor kept outside the integral, `variables` lists the variables to integrate from $[0, \infty)$, and `coefficients` collects all symbolic

parameters treated as constants during integration. ✧

In the next section, we provide the necessary theoretical background to understand the tropical subtraction algorithm and the hyperlogarithm integration procedure that underlie `STIntegrate`.

2.4 Numerical evaluation and verification

`SubTropica` also allows one to numerically evaluate integrals (linearly reducible or not) and verify an analytic output.

In particular, `STNIntegrate` numerically evaluates integrals by interfacing with `pySecDec` [31], `FIESTA` [32], `AMFlow` [33], or `feyntrop` [56] as backends. It accepts all the same input formats as `STIntegrate` (Forms 1–3), including generic parametric representations. This is particularly useful for integrals that are not linearly reducible, where a symbolic evaluation via `STIntegrate` is not available. For instance, the two-loop generic masses (elliptic) kite

```
edges = {{{1, 2}, m1}, {{1, 3}, m2}, {{1, 4}, m3},
         {{2, 3}, m4}, {{2, 4}, m5}};
nodes = {{3, M}, {4, M}};
STNIntegrate[{edges, nodes},
  "Substitutions" -> {m1 -> 67/23, m2 -> 59/31, m3 -> 159/31,
                      m4 -> 59/131, m5 -> 117/137, M -> 31 I/5}]
```

returns $-0.0708835 + \mathcal{O}(\varepsilon)$ in about 1.2 seconds using `pySecDec`.

The function `STVerify` cross-checks a symbolic result from `STIntegrate` against a numerical evaluation. If it exists, it automatically generates a Euclidean kinematic point, runs `pySecDec` (or `FIESTA`/`AMFlow`/`feyntrop`), evaluates the symbolic expression numerically via `ginsh`, and compares the Laurent coefficients order by order. For example,

```
diag = {{{{1,2},0},{{1,2},0},{{2,3},0},{{3,4},0},
         {{3,4},0},{{1,4},0}}, {{1,0},{2,0},{3,0},{4,0}}};
result = STIntegrate[diag];
STVerify[diag, result]
```

produces a coefficient-by-coefficient comparison and returns `"pass" -> True` with a maximum relative error of $\sim 10^{-5}$.

As mentioned above, `STNIntegrate` also accepts parametric integrands, using the same calling conventions as `STIntegrate` (Form 3). For the diagram `diag` in the

last example, the Schwinger-parameter representation (1.4) (with $x_4 = 1$) can be evaluated as follows:

```
(* Symanzik polynomials of the last "diag" above*)
U = x1 x2 + x1 x3 + x2 x3 + x1 x5 + x2 x5 + x1 x2 x5
    + x1 x3 x5 + x2 x3 x5 + x1 x6 + x2 x6 + x1 x5 x6 + x2 x5 x6;
F = s23 x1 x2 x5 + s12 (x1 x3 x6 + x2 x3 x6 + x1 x3 x5 x6 + x2 x3 x5 x6);
pref = Exp[3 eps EulerGamma] Gamma[3 eps];
integrand = U^(2(2 eps-1)) (-F)^(-3 eps);
subs = "Substitutions" -> {s12 -> -7/31, s23 -> -43/89};

(* [0,∞): bare symbols *)
STNIntegrate[pref integrand, x1, x2, x3, x5, x6, subs]
(* Out: 0.3333/eps^3 + 4.155/eps^2 + 32.36/eps + 183.7 + O[eps] *)

(* [0,1]: explicit bounds (for illustration) *)
STNIntegrate[pref integrand, {x1,0,1}, {x2,0,1}, {x3,0,1},
                             {x5,0,1}, {x6,0,1}, subs]
(* Out: 0.2195/eps^2 + 2.987/eps + 22.59 + O[eps] *)

(* Pre-built tuple: {prefactor, integrand, vars, coeffs} *)
STNIntegrate[{pref, integrand, {x1,x2,x3,x5,x6}, {s12,s23}}, subs]
(* same output as bare-symbol call *)
```

2.5 SubTropica library

SubTropica comes with a comprehensive open-source library of diagrams, including references to the most relevant papers that featured them. The library also stores results of computations obtained with SubTropica. Its online version can be accessed at

subtropi.ca

It is continuously updated with new diagrams and results. We welcome new submissions to the library which can be sent to us simply by clicking the "Submit to SubTropica library" button after a successful integration, or alternatively by executing

```
STSubmitResult[]
```

after a successful computation.

3 Theory and algorithms

We begin by defining Euler integrals and their ε-expansion, then introduce the Newton polytope machinery that governs the singularity structure. The Nilsson–Passare continuation and tropical subtraction algorithms convert a divergent Euler integral into a sum of locally finite terms, which are then integrated one variable at a time using the hyperlogarithm calculus implemented in `HyperIntica` (and `HyperInt` [57]). We close this section with a discussion of linear reducibility, the key condition that ensures that this sequential integration terminates in closed form.

3.1 An invitation to Euler integrals

Mathematically, an Euler integral can be defined as the Mellin transform of a rational function,

$$I(\mathbf{s}, c) = \int_{\mathbb{R}^n_{\geqslant 0}} \frac{\mathrm{d}\alpha_1}{\alpha_1} \dots \frac{\mathrm{d}\alpha_n}{\alpha_n} \times \left(\prod_{j=1}^{m} P_j(\alpha, \mathbf{s})^{c_j} \right) , \tag{3.1}$$

where $c_j \in \mathbb{C}$ and P_j are all polynomials in the integration variables $\alpha = (\alpha_1, \dots, \alpha_n)$ with coefficients collectively denoted $\mathbf{s}$.

An Euler integral is a meromorphic function of the exponent variables c [1]. On the other hand, the dependence on the coefficient variables $\mathbf{s}$ is much more complicated, involving sophisticated generalizations of the logarithm, which we discuss later in this section (see Sec. 3.3).

To streamline the discussion in this section, we will restrict ourselves to the case where the exponent variables depend on a common real parameter ε as[2]

$$c_j = c_j(\varepsilon) = a_j + b_j \varepsilon \, , \tag{3.2}$$

where a_j, b_j are fixed complex numbers. In quantum field theory, the parameter ε is often called the *dimensional regulator*. While the subtraction still applies in the case of general a_j, the integration discussed below needs integer a_j.

With this parametrization, the Euler integral becomes a function $I(\mathbf{s}, \varepsilon)$ of finitely many coefficient variables $\mathbf{s}$ and a single real parameter ε. It turns out that $I(\mathbf{s}, \varepsilon)$

[2] In the examples section below (see Sec. 4.3.2), we will encounter integrals for which more than one such parameters appear, and show how to deal with them using `SubTropica`.

extends to a meromorphic function of ε. We have an expansion around $\varepsilon = 0$,

$$I(\mathbf{s}, \varepsilon) = \sum_{i=-N}^{\infty} \varepsilon^i I^{(i)}(\mathbf{s}) \,. \tag{3.3}$$

In this package, we provide an algorithmic and practical way to find an expansion for the coefficient functions $I^{(i)}(\mathbf{s})$ in terms of convergent integrals that can be evaluated with the algorithm implemented in the MAPLE program `HyperInt` [57] and the new MATHEMATICA package `HyperIntica` shipped with `SubTropica`.

The strategy we follow is based on the *tropical subtraction formula* presented in [1, Sec. 6] reviewed below (see also [2, 3]). The idea is to add and subtract "counter-terms" from the original integrand, according to a combinatorial pattern that reflects the boundary structure of a polytope attached to the integral. Once the dust settles, we are left with an expansion of the Euler integral,

$$I(\mathbf{s}, \varepsilon) = \sum_i \phi_i(\varepsilon) I_i(\mathbf{s}, \varepsilon) \,, \tag{3.4}$$

as a combination of *locally finite* integrals $I_i(\mathbf{s}, \varepsilon)$. By definition, these are integrals whose ε-expansion can be performed directly at the integrand level, providing the desired integral representation for the coefficients $I^{(i)}(\mathbf{s})$.

The tropical subtraction formula is valid only for integrals that satisfy a certain *geometric property*. Luckily, any integral can be written as a combination of such, using the *Nilsson–Passare* analytical continuation [58], or tropical continuation for brevity. In fact, tropical continuation can be used to expand an integral directly in locally finite ones. Therefore it provides an alternative to tropical subtractions [5]. However this usually results in large complicated expressions, which limits the usefulness of this approach. Nevertheless, even when expanding a divergent integral in locally finite ones is prohibitive, limiting the tropical continuation to expand in integrals that merely satisfy the geometric property, rather than being locally finite, is often manageable.

In summary, chaining tropical continuation and tropical subtraction gives an efficient algorithm to expand an arbitrary Euler integral in terms of locally finite ones. This is the core functionality of `SubTropica`.

A comment on positivity. The treatment of Euler integrals simplifies drastically if each polynomial appearing in the integrand is sign-coherent, that is if

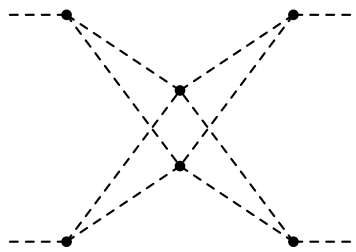

Figure 2: The crown diagram features a non-sign-definite Symanzik polynomial.

its coefficients all have the same sign.[3] In this case the Euler integrand does not vanish in the interior of the domain of integration. A crucial consequence is that the singularities of the Euler integral (poles in ε) arise only from the *boundary* of the integration domain. This is where tropical geometry enters the game: the *Newton polytope* of the integrand provides the only systematic way to detect these singular corners.

In the rest of this section, we will discuss two methods to expand an Euler integral in terms of locally finite ones. Both algorithms are rigorously valid only in the case of sign-coherent polynomials. If this property is not satisfied, the Euler integral may have additional poles in ε that are *not* captured by these methods. This is a famous issue that plagues all direct-integration techniques and their implementation in public software such as `asy` [60], `FIESTA` [32] or `pySecDec` [31].

Currently, the only safe way to deal with this problem (and detect otherwise "hidden" singularities) is to perform a careful Landau analysis: one has to enumerate the irreducible components of the Landau variety which have positive solutions. Although this can be done algorithmically with publicly available software, such as `Macaulay2` [61] and `Bertini` [62], it is computationally much more expensive than the tropical analysis discussed here. Therefore, this is to this day no generic practical solution (see, e.g., [63, 64] for related discussions).

In physical applications, this issue arises only in certain advanced calculations involving highly non-planar massless diagrams, such as the crown diagram in Fig. 2, which is connected to the problem of "Glauber modes" (singularities that do not fall into the standard soft-collinear classification) [64]. The current version of `SubTropica` does not handle these divergences and a systematic treatment is left to future work.

[3] This requirement can, in fact, be relaxed to allow polynomials that are *completely non-vanishing* [59].

3.2 Tropical algorithms

In this section, we describe how to use rudiments from tropical geometry to detect and calculate divergences of Euler integrals.

3.2.1 Divergences

The first step is to understand how divergences originate.

A warming-up example. Let us consider a simple toy example,

$$A = \int_0^\infty \int_0^\infty \mathrm{d}\alpha_1 \, \mathrm{d}\alpha_2 \, \alpha_1^{1+\varepsilon} \alpha_2^{\varepsilon} \left(\alpha_1^2 + \alpha_2 + \alpha_1\alpha_2\right)^{-(2+\varepsilon)} \, . \tag{3.5}$$

The singular behavior of the integrand around the origin suggests that the integral may develop a divergence there. In order to expose it, we perform the rescaling

$$\alpha_2 \to \alpha_1^2 \alpha_2 \, . \tag{3.6}$$

The integral transforms to

$$A = \int_0^\infty \int_0^\infty \mathrm{d}\alpha_1 \, \mathrm{d}\alpha_2 \, \alpha_1^{-1+\varepsilon} \alpha_2^{\varepsilon} \left(1 + \alpha_2 + \alpha_1\alpha_2\right)^{-(2+\varepsilon)} \, . \tag{3.7}$$

Note how the singular behavior of the integrand is completely captured by the *monomial* $\alpha_1^{-1+\varepsilon}$: the polynomials resulting from the rescaling are non-vanishing at the origin. This allows to neatly extract a pole in ε. Integrating around $\alpha_1 < 1$ (which corresponds to a small neighborhood of the origin in the original coordinates) gives

$$\begin{aligned} &\int_0^1 \int_0^\infty \mathrm{d}\alpha_1 \, \mathrm{d}\alpha_2 \, \alpha_1^{-1+\varepsilon} \alpha_2^{\varepsilon} \left(1 + \alpha_2 + \alpha_1\alpha_2\right)^{-(2+\varepsilon)} = \\ &= \int_0^1 \int_0^\infty \mathrm{d}\alpha_1 \mathrm{d}\alpha_2 \left(\alpha_1^{-1+\varepsilon} \alpha_2^{\varepsilon} (1+\alpha_2)^{-(2+\varepsilon)}\right) \left(1 + \mathcal{O}(\alpha_1)\right) \\ &= \frac{1}{\varepsilon} \times \int_0^\infty \mathrm{d}\alpha_2 \alpha_2^{\varepsilon} (1+\alpha_2)^{-(2+\varepsilon)} + \ldots \end{aligned} \tag{3.8}$$

The crucial step was to consider the rescaling (3.6), whose origin is subtler than it seems. For example, consider the similar rescaling $\alpha_2 \to \alpha_1\alpha_2$. As $\alpha_1 \to 0$, it

does the same job as (3.6) of "blowing-up" the origin, but now we have

$$A \to \int_0^\infty \int_0^\infty \mathrm{d}\alpha_1 \mathrm{d}\alpha_2 \; \alpha_1^\varepsilon \, \alpha_2^\varepsilon \, (\alpha_1 + \alpha_2 + \alpha_1\alpha_2)^{-(2+\varepsilon)} \;, \tag{3.9}$$

so that integrating around $\alpha_1 \sim 0$ does not produce a pole in ε.

Tropicalization. Let us now formulate the problem in more general terms. Consider an Euler integral

$$\int_{\mathbb{R}^n_{\geqslant 0}} \frac{\mathrm{d}\alpha_1}{\alpha_1} \cdots \frac{\mathrm{d}\alpha_n}{\alpha_n} \, \mathcal{I}(\alpha_1, \ldots, \alpha_n) \,, \tag{3.10}$$

and an arbitrary rescaling

$$\alpha_i \to \alpha_i \lambda^{-\rho_i}, \quad \rho = (\rho_1, \ldots, \rho_n) \in \mathbb{Q}^n \,, \tag{3.11}$$

Note that we can always eliminate one of the original variables α_i in favor of λ, and thus regard (3.11) as a change of coordinates. With this understanding the limit $\lambda \to 0^+$ probes a corner of the boundary of the integration space, where the integration variables scale as (3.11). Clearing denominators and factoring polynomials, we can pull out an overall monomial in λ,

$$\mathcal{I}(\alpha_i \lambda^{-\rho_i}) = \lambda^{-\mathrm{Trop}\ \mathcal{I}(\rho)} \mathcal{J}(\alpha, \lambda) \,. \tag{3.12}$$

We use (3.12) to define the *tropical integrand*, Trop $\mathcal{I}$, a function that associates to the scaling vector ρ minus the exponent of this overall monomial. Keeping in mind the overall logarithmic differential form, which is invariant under the transformation, we see that

$$\mathrm{Re}\,\mathrm{Trop}\ \mathcal{I}(\rho)|_{\varepsilon=0} < 0 \quad \text{for all } \rho \in \mathbb{Q}^n \qquad \Rightarrow \qquad A = \mathcal{O}(\varepsilon) \,. \tag{3.13}$$

Due to (3.2), the tropical integrand is always at most linear in ε,

$$\mathrm{Trop}\ \mathcal{I}(\rho) = a + b\,\varepsilon \,, \tag{3.14}$$

depending on the value of a we say that ρ is either convergent ($\mathrm{Re}\, a < 0$), logarithmically divergent ($\mathrm{Re}\, a = 0$) or power divergent ($\mathrm{Re}\, a > 0$).

As it stands, the criterion (3.13) is not particularly helpful, because it requires checking the sign of the tropical integrand over infinitely many vectors ρ. Luckily,

it turns out that one only needs to evaluate the tropical integrand on a finite set of "critical" vectors [65]. These are the normals to the facets of a polytope canonically attached to the integrand $\mathcal{I}$.

Newton Polytopes. The Newton polytope of a polynomial is the convex hull of the exponents of its monomials [66]. In other words, if

$$P = \sum_{\mathbf{m}\in\mathbb{Z}^n} s_{\mathbf{m}} \prod_{i=1}^{n} \alpha_i^{m_i} \,, \tag{3.15}$$

then the Newton polytope Newt P is the convex hull

$$\text{Newt } P \equiv \text{Conv}\{\mathbf{m} \in \mathbb{Z}^n \mid s_{\mathbf{m}} \neq 0\} \,. \tag{3.16}$$

For an Euler integrand,

$$\mathcal{I}(\alpha) = \prod_j P_j(\alpha)^{a_j + b_j \varepsilon} \,, \tag{3.17}$$

we define its Newton polytope to be Newt $\mathcal{I} \equiv$ Newt $\prod_j P_j$.

The convex hull description (3.16) is known as the *vertex presentation* of the polytope. Every polytope admits a dual description, or *facet presentation*, as an intersection of half-spaces,

$$\text{Newt } P = \bigcap_j \{\alpha = (\alpha_1, \ldots, \alpha_n) \mid d_j - \rho_j \cdot \alpha \geqslant 0\}. \tag{3.18}$$

for some constants $d_j \in \mathbb{Q}$ and vectors $\rho_j \in \mathbb{Q}^n$.

Suppose that the locus where one of inequalities of (3.18) is saturated is a co-dimension one boundary of Newt P, also called a *facet*. Then the corresponding vector ρ_j can be interpreted geometrically as the outer normal to the facet.

The collection of all the normals to the facets is a finite set of "critical" vectors that control the singularities of the integral. If these vectors are convergent (using the terminology introduced below (3.13)), then the Euler integral does not have poles in ε.

In the case of Newton polytopes, the vertex description (3.16) is immediately available from the knowledge of the polynomial (3.15). The facet description, however, requires some work. The problem involves only linear algebra, but it

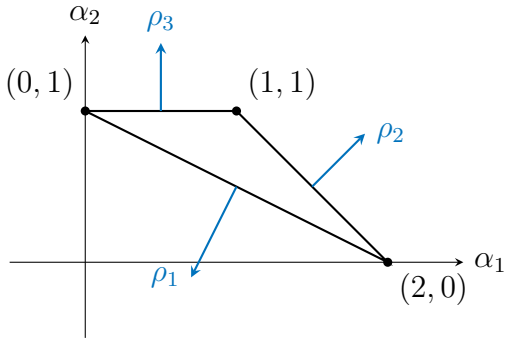

Figure 3: The Newton polytope describing the singularities of the Euler integral (3.5)

can be computationally very demanding, depending on the underlying combinatorics. Fortunately, efficient and sophisticated solutions to this problem are implemented in various software. In `SubTropica` we outsource this computational task to `polymake` [51, 52].

Example. Let us illustrate the notions introduced so far by revisiting the example (3.5). The Euler integrand is

$$\mathcal{I} = \alpha_1^{2+\varepsilon}\alpha_2^{1+\varepsilon}\left(\alpha_1^2 + \alpha_2 + \alpha_1\alpha_2\right)^{-(2+\varepsilon)} , \tag{3.19}$$

it tropicalizes to

$$\text{Trop}\,\mathcal{I} = (2+\varepsilon)\alpha_1 + (1+\varepsilon)\alpha_2 - (2+\varepsilon)\max(2\alpha_1, \alpha_2, \alpha_1+\alpha_2) , \tag{3.20}$$

Note the slight abuse of notation, where we keep using the same variables, α_i, as arguments of both $\mathcal{I}$ and Trop $\mathcal{I}$.

Up to translation, the Newton polytope Newt $\mathcal{I}$ is the triangle of Fig. 3. At first glance, it is evident that its normals are $\rho_1 = (-1,-2)$, $\rho_2 = (1,1)$, and $\rho_3 = (0,1)$. Plugging these into (3.20) it is easy to check that ρ_1 is logarithmically divergent, with Trop $\mathcal{I}(\rho_1) = \varepsilon$, while both ρ_2 and ρ_3 are convergent. The rescaling (3.11) associated to ρ_1 is

$$\alpha_1 \to \alpha_1\lambda \quad \alpha_2 \to \alpha_2\lambda^2 , \tag{3.21}$$

which is of course equivalent to the previously mysterious (3.6).

We have now understood how Newton polytopes encode the singularities of an

Euler integral. The poles in the dimensional regulator ε arise from the corners of the integration space exposed by the rescaling (3.11). The exponent vector defining the rescaling is an outer-pointing normal to a facet of the polytope.

3.2.2 Locally finite expansion

We now turn to the expansion of a divergent Euler integral in terms of locally finite integrals, Eq. (3.4). As we will see shortly, the Newton polytope of the Euler integral once again plays a central role: it encodes in its boundary structure the data necessary to construct the expansion.

Tropical continuation. Consider an Euler integral $I(\varepsilon)$ and suppose that ρ is a divergent vector. Consider the shifted Euler integral

$$I(\lambda;\varepsilon) = \int_{\mathbb{R}^n_{\geqslant 0}} \frac{\mathrm{d}\alpha_1}{\alpha_1} \dots \frac{\mathrm{d}\alpha_n}{\alpha_n} \mathcal{I}(\alpha_i \lambda^{-\rho_i};\varepsilon)\,, \tag{3.22}$$

which reduces to the original integral $I(\varepsilon)$ for $\lambda = 1$. In fact, $\frac{\mathrm{d}}{\mathrm{d}\lambda} I(\lambda;\varepsilon) = 0$, because the dependence on λ can be eliminated at the integrand level by the rescaling $\alpha \to \alpha\lambda^\rho$ which leaves invariant both the measure and the integration domain.[4] We have

$$0 = \int_{\mathbb{R}^n_{\geqslant 0}} \frac{\mathrm{d}\alpha_1}{\alpha_1} \dots \frac{\mathrm{d}\alpha_n}{\alpha_n} \left(\frac{\mathrm{d}}{\mathrm{d}\lambda} \mathcal{I}(\alpha_i \lambda^{-\rho_i},\varepsilon) \right) \bigg|_{\lambda=1} = -I(\varepsilon) \mathrm{Trop}\, \mathcal{I}(\rho) + \sum_i I_i(\varepsilon)\,,$$

or equivalently,

$$I(\varepsilon) = \frac{1}{\mathrm{Trop}\, \mathcal{I}(\rho)} \left(\sum_i I_i(\varepsilon) \right)\,. \tag{3.23}$$

Here, $I_i(\varepsilon)$ are new Euler integrals that arise by the action of $\frac{\mathrm{d}}{\mathrm{d}\lambda}$ on $\mathcal{J}_\rho(\alpha,\lambda)$ in (3.12). It turns out that the integrands $\mathcal{I}_i$ for the integrals I_i satisfy $\mathrm{Trop}\, \mathcal{I}_i \leqslant \mathrm{Trop}\, \mathcal{I}$, with the inequality being strict when both sides are evaluated on the ray ρ. This can be seen by considering how the facet ρ of the Newton polytope of $\mathcal{I}(\alpha\lambda^{-\rho};\varepsilon)$ is "shaved" by the action of $\frac{\mathrm{d}}{\mathrm{d}\lambda}$.

By iterating this procedure sufficiently many times, we can express $I(\varepsilon)$ as a linear combination of locally finite Euler integrals multiplied by explicit poles in ε. This

[4] In this context, λ should be thought of as a parameter, not an integration variable.

is the tropical continuation due to Nilsson-Passare [58, 67, 68]. While it provides a systematic approach to compute the ε-expansion of an Euler integral, tropical continuation is computationally very expensive. This can be traced to two main bottlenecks:

1. Iterating derivatives produces an exponential number of integrals.
2. The pole part $I(\varepsilon)$ is presented in terms of integrals with the same dimension as $I(\varepsilon)$.

We will next describe an alternative strategy which overcomes these limitations. It uses more elaborate information about the boundary structure of the Newton polytope: not only which are the facets responsible for divergences, but also how they intersect in lower dimensional faces of the polytope.

Geometric property. Let $\mathcal{P}$ be a polytope, and suppose that a subset of its facets, $\{\mathcal{F}_i,\ i = 1, \ldots, n\}$, share a common vertex. Then we say that these are *compatible* facets.

If divergent facets are compatible, they can conspire to produce higher order poles. In this sense, the Newton polytope leaves an imprint of itself in the pole structure of an Euler integral. To capture this pattern we construct the set of all collections of compatible, divergent (normals to the) facets,

$$\Sigma^{\text{div}} = \{\sigma | \sigma = (\rho_1, \ldots, \rho_m),\ \rho_i \text{ are divergent, compatible normals}\} \cup \{\varnothing\}. \quad (3.24)$$

Note that we conventionally include the empty set, to streamline formulae down the line.

Consider now an Euler integrand $\mathcal{I}$ with at most logarithmic divergences. We say that $\mathcal{I}$ satisfies the *geometric property* if for every divergent vector ρ we are able to find a vector w_ρ such that $w_\rho \cdot \rho' = -\delta_{\rho,\rho'}$, for all divergents normals ρ' that are compatible with ρ.

The geometric property has an important consequence. The set Σ^{div} is an abstract simplicial complex. Furthermore, if $\sigma = (\rho_1, \ldots, \rho_m) \in \Sigma^{\text{div}}$, then the normals ρ_i are linearly independent and the corresponding facets meet in a face of codimension m,

In general, Euler integrals do not satisfy the geometric property nor have only logarithmic singularities. However, we can always tropically continue along all vectors ρ which are either power-divergent or do not admit an accompanying vector

w_ρ. By iterating the procedure sufficiently many times, one is eventually left with logarithmically divergent integrals that satisfy the geometric property.

Our interest in integrals that satisfy the geometric property is due to the existence of an algorithm that allows to expand them in terms of locally finite integrals, and which is substantially more efficient than tropical continuation. This will be discussed next.

Tropical subtraction. The main idea is to add and subtract suitable "counter-terms" $\mathcal{I}^{\mathrm{ct}}$ from a divergent Euler integrand. The counter-terms must be chosen so that they cancel the divergences of $\mathcal{I}$, while being simple enough that they can be integrated as many times as the order of the pole they cancel. In other words, the goal is to write

$$\mathcal{I} = \left[\mathcal{I} - \mathcal{I}^{\mathrm{ct}}\right] + \mathcal{I}^{\mathrm{ct}}\,, \tag{3.25}$$

in such a way that the "renormalized" integrand

$$\mathcal{I}^{\mathrm{ren}} \equiv \mathcal{I} - \mathcal{I}^{\mathrm{ct}}\,, \tag{3.26}$$

is locally finite, and the integral

$$\int_{\mathbb{R}^n_{\geqslant 0}} \frac{\mathrm{d}\alpha_1}{\alpha_1} \cdots \frac{\mathrm{d}\alpha_n}{\alpha_n}\, \mathcal{I}^{\mathrm{ct}}\,, \tag{3.27}$$

can be partially integrated, dropping one dimension for each pole in ε exposed.

If $\mathcal{I}$ is an integrand that satisfies the geometric property, and with at most logarithmic divergences, then we can follow [1] to construct such counter-terms. The combinatorics of the subtraction formula (3.25) is controlled by the set Σ^{div} from (3.24). It reflects the intersection pattern of the facets of Newt $\mathcal{I}$ with divergent normal vectors ρ.

The counter-terms are the product of two factors.

The first is the *restriction* of the integrand on a face (also known as *initial form* in the context of tropical and toric geometry [69]). It is the integrand obtained by keeping from each polynomial P only the monomials that lie on a particular face of Newt P. The fact that the integrand and its restriction have similar Newton polytopes, and thus matching singularities, is what makes them natural candidates in an ansatz for the counter-terms.

Unfortunately, restrictions alone cannot be used as counter-terms. The reason is that they introduce singularities in addition to the ones that they cancel. This can be easily seen already in the simplest case of one-dimensional integrals. Their Newton polytopes are necessarily always segments. Integrands with a segment and its vertices for Newton polytopes cannot be combined in a locally finite combination.

In order to address this issue, we introduce the second factor of the counter-term. To any vector ρ, we associate the function

$$v_\rho = \frac{1}{1 + \prod_i \alpha_i^{w_i}} \,. \tag{3.28}$$

To an element $(\rho_1, \ldots, \rho_m) \in \Sigma^{\text{div}}$ we associate the function

$$v_{(\rho_1,\ldots,\rho_m)} = \prod_i v_{\rho_i}^{1-\text{Trop } \mathcal{I}(\rho_i)} \,. \tag{3.29}$$

We can finally construct the counter-terms. For any $\sigma \in \Sigma^{\text{div}}$ we define the counter-term

$$\mathcal{I}_\sigma = v_\sigma \times \mathcal{I}|_\sigma \,, \tag{3.30}$$

where $|_\sigma$ denotes the restriction on the intersection of the compatible facets with normals $\rho_{1,\ldots,m}$. We also set $\mathcal{I}_\varnothing = \mathcal{I}$.

For any counter-term integrand $\mathcal{I}_\sigma$ we define a renormalization map

$$\mathcal{I}_\sigma \to \mathcal{I}_\sigma^{\text{ren}} = \sum_{\sigma' \in \Sigma^{\text{div}} | \sigma \subseteq \sigma'} (-1)^{|\sigma| - |\sigma'|} \mathcal{I}_{\sigma'}. \tag{3.31}$$

A simple application of the inclusion-exclusion principle[5] gives

$$\mathcal{I} = \sum_\sigma \mathcal{I}_\sigma^{\text{ren}} \,. \tag{3.32}$$

We now make two important claims. Further details can be found in [1]. First, the renormalized integrand $\mathcal{I}_\varnothing^{\text{ren}}$ is locally finite. Second, the terms in (3.32) corresponding to $\sigma \neq \varnothing$ can be integrated exactly $|\sigma|$ times, giving a pole of this

[5] More precisely, this follows from the Möbius inversion formula applied on the poset $(\Sigma^{\text{div}}, \supset)$, see [70].

order.

This results in the tropical subtraction formula,

$$\int_{\mathbb{R}^n_{\geqslant 0}} \frac{\mathrm{d}\alpha_1}{\alpha_1} \dots \frac{\mathrm{d}\alpha_n}{\alpha_n} \mathcal{I} = \sum_{\sigma} \mathrm{Vol}(\sigma) \int_{V_\sigma} \frac{\mathrm{d}\alpha_1}{\alpha_1} \dots \frac{\mathrm{d}\alpha_n}{\alpha_n} \bigg|_{V_\sigma} \prod_{i=1}^{n} \alpha_i^{u_{\sigma,i}} (\mathcal{I}|_\sigma)^{\mathrm{ren}} \,, \tag{3.33}$$

where all integrals are locally finite. The integration of the counter-term $\mathcal{I}_\sigma$ localizes to a subpace V_σ where certain variables α_i are set to 1. This localization produces poles that are contained in the prefactor, $\mathrm{Vol}(\sigma)$. An additional monomial, with exponent vector u_σ, is generated in the process.

The subspace V_σ is not unique. It can be constructed as follows. Let $\mathbf{e}_i$ be the coordinate vectors of $\mathbb{R}^n$. Choose a subset $\eta = \{e_j\}_{j\in J}$ of $(n-m)$ coordinate vectors that completes the vectors $\sigma = \{\rho_1, \dots, \rho_m\}$ to a basis of $\mathbb{R}^n$. Let $M_\sigma = \sigma|\eta$ be the matrix having as columns the vectors of σ followed by those of η. With this choice the subspace is

$$V_\sigma = \{\alpha_j \geqslant 0\}_{j\in J} \cap \{\alpha_i = 1\}_{i\notin J} \,, \tag{3.34}$$

and the prefactor is

$$\mathrm{Vol}(\sigma) = |\det(M_\sigma)| \prod_{\rho\in\sigma} \frac{1}{-\mathrm{Trop}\ \mathcal{I}(\rho)} \,. \tag{3.35}$$

Finally, to compute the exponent vector u_σ, consider the map $\pi_J : \mathbb{R}^n \to \mathbb{R}^n$ that sets to zero the entries not in J. Then

$$u_\sigma = \sum_{\rho\in\sigma} \mathrm{Trop}\ \mathcal{I}(\rho) \pi_J(w_\rho) \,. \tag{3.36}$$

Summary of the main algorithm. The algorithms described in this section provide a systematic way to obtain the expansion (3.4) of an arbitrary Euler integral. The procedure requires three steps.

1. Compute the polytope Newt $\mathcal{I}$, enumerate its divergent facets, and construct the set Σ^{div} from (3.24).
2. Apply Nilsson–Passare tropical continuation on power divergent vectors, and on sufficiently many vectors that violate the geometric property.
3. Compute the tropical subtraction formula for each of the resulting integrals.

Local finiteness ensures that we can expand the integrands in ε and integrate term-by-term the resulting convergent integrals. These can then be integrated with the algorithm by Brown and Panzer, which will be reviewed in Sec. 3.3.

3.2.3 A worked out example

Let us conclude this section with a simple example where we can see all the tropical algorithms introduced so far at work.

Consider the Euler integral

$$A(\varepsilon) = \int_{\mathbb{R}^2_{\geqslant 0}} \frac{\mathrm{d}x_1}{x_1}\frac{\mathrm{d}x_2}{x_2} x_1^{\varepsilon} x_2^{\varepsilon} (1 + x_1 + x_2)^{-3\varepsilon}. \tag{3.37}$$

The first step is to perform the tropical analysis of the singularities of the integrand.

In `SubTropica` this can be done automatically, as follows:

```
integrand = x1^eps x2^eps (1 + x1 + x2)^(-3 eps);
xvars = {x1, x2};
coeffs = {};

(* Compute Newton polytope, tropical function, and divergent fan *)
analysis = STPreAnalysis[integrand,xvars,coeffs];

(* Outer normals of Newt[I] *)
vectors = analysis["trData"]["rays"]
(* Out: {{-1,0},{0,-1},{1,1}} *)

(* Values of Trop[I] on the normals *)
tropicalValues = analysis["trops"]
(* Out: {-eps, -eps, -eps} *)

(* Divergent fan *)
sigmaDiv = analysis["faces"]
(* Out: {{}, {1}, {2}, {3}, {1, 2}, {1, 3}, {2, 3}} *)

(* Functions v_ρ *)
vFactors = analysis["us"]
(* Out: {"NotFound", "NotFound", "NotFound"} *)
```

This prints the outer normals ρ_i of Newt $\mathcal{I}$, the values that Trop $\mathcal{I}$ takes on them, the set Σ^{div} (with vectors ρ_i indicated by the label i) and the functions v_ρ from

(3.28), if it is possible to find them. From the output we learn that all the normals are logarithmically divergent and pairwise compatible. From the explicit vectors, it is also easy to see that they all violate the geometric property. Indeed no function v_ρ is found.

We proceed with the second step, for instance by applying the Nilsson–Passare tropical continuation on $\rho = \{1, 1\}$:

```
continuation = STTropicalContinuation[{{1,integrand}},xvars,{{1,1}}]
(* Out: {{3, x1^eps x2^eps (1 + x1 + x2)^(-1-3 eps)}} *)
```

which represents the identity

$$A(\varepsilon) = 3 \int_{\mathbb{R}^2_{\geqslant 0}} \frac{\mathrm{d}x_1}{x_1} \frac{\mathrm{d}x_2}{x_2} \mathcal{J}(x_1, x_2), \quad \mathcal{J}(x_1, x_2) = x_1^\varepsilon x_2^\varepsilon (1 + x_1 + x_2)^{-(1+3\varepsilon)} . \tag{3.38}$$

Repeating the tropical analysis for the new integrand we now find the following functions v_ρ

```
STPreAnalysis[continuation[[1,2]], xvars, coeffs]["us"]
(* Out: {x1/(1+x1), x2/(1+x2)} *)
```

This means that we can compute the tropical subtraction without further use of Nilsson–Passare continuation:

```
newintegrand = 3 x1^eps x2^eps (1 + x1 + x2)^(-1-3 eps);
subtraction = STTropicalSubtraction[newintegrand,xvars,coeffs]
```

This returns the right-hand-side of (3.33), as a list of pairs $\{\mathrm{Vol}(\sigma)\alpha^{u_\sigma}(\mathcal{I}|_\sigma)^{\mathrm{ren}}, V_\sigma\}$ indexed consistently with Σ^{div}. For example, the term labelled by $\sigma = \{1\}$ is stored in

```
subtraction[[2]]
(* Out: {{x2^(-1+eps) (1+x2)^(-1-3 eps)/eps, -x2^(-1+eps) (1/(1+x2))^(1+eps)/eps}, {x2}} *)
```

It represents a renormalized integrand (3.31), whose counter-terms are stored as a list. Note the presence of an explicit pole in ε, which comes from the prefactor $\mathrm{Vol}(\sigma)$. The expression is to be integrated over $x_2 \in [0, \infty]$.

We end by remarking that the entire procedure is automatized in a single function:

```
STExpandIntegral[integrand,xvars,coeffs]
```

3.3 Integration of locally finite integrands

The purpose of this section is to briefly review the by-now well-understood strategy of [5, 71] (first applied in [72–75]) for evaluating locally finite expanded Euler integrals obtained from the procedure discussed in Sec. 3.2. We will further restrict ourselves to the cases for which these integrals are of the form

$$f_n = \int_0^\infty \cdots \int_0^\infty f_{n-1}(x_1, \ldots, x_n)\, \mathrm{d}x_1 \cdots \mathrm{d}x_n \,, \tag{3.39}$$

where f_{n-1} is assumed to be expressible as a product of rational functions and hyperlogarithms (defined below in (3.40) and discussed further in App. B). Other integration domains $[a, b]$ can be accommodated through the change of variables $x \mapsto t = \frac{a+bx}{1+x}$.

In cases where the linear reducibility condition discussed later in Sec. 3.4 is satisfied, then the integral f_n can be evaluated by integrating one variable at a time. One efficient method to achieve this rests on two key structural properties of hyperlogarithms: their simple behavior under differentiation, which systematizes the construction of primitives/antiderivatives, and the fact that they form a shuffle algebra. The latter property allows one to reorganize products of hyperlogarithms into canonical combinations and, in particular, to "shuffle regularize" hyperlogarithms by isolating unambiguously the finite contributions of antiderivatives at (potentially) singular integration boundaries.

We summarize here how these elements combine to yield a complete integration procedure.

3.3.1 Hyperlogarithms and their derivatives

Hyperlogarithms are defined as iterated integrals

$$\mathrm{H}(z; \sigma_1, \ldots, \sigma_w) \equiv \int_0^z \frac{\mathrm{d}t_1}{t_1 - \sigma_1} \int_0^{t_1} \frac{\mathrm{d}t_2}{t_2 - \sigma_2} \cdots \int_0^{t_{w-1}} \frac{\mathrm{d}t_w}{t_w - \sigma_w} \,, \tag{3.40}$$

with the convention $\mathrm{H}(z; \varnothing) = 1$. The sequence $\vec{\sigma} = [\sigma_1, \ldots, \sigma_w]$ is called a *word*, and the integer w is its (transcendental) *weight*.

The entire integration algorithm for the f_i's follows from the action of the total differential. Differentiating with respect to the upper limit z yields

$$\frac{\mathrm{d}}{\mathrm{d}z} \mathrm{H}(z; \sigma_1, \ldots, \sigma_w) = \frac{1}{z - \sigma_1} \mathrm{H}(z; \sigma_2, \ldots, \sigma_w) \,. \tag{3.41}$$

This relation encodes the fundamental point: differentiation peels off the first letter σ_1, while integration prepends a new letter. When the letters σ_i themselves depend on a parameter, the total differential acquires additional contributions from the variation of each letter (see Eq. (3.3.32) in [76] for the complete formula). However, the core principle remains unchanged: the differential structure of hyperlogarithms is entirely determined by the rational one-forms $\mathrm{d}\log(z-\sigma)$.

3.3.2 The integration strategy

Suppose now that we wish to integrate the variable x_n from 0 to ∞, given an integrand f_{n-1} that has already been expressed in terms of hyperlogarithms. The strategy proceeds in three steps.

In the first step, the integrand is written as a linear combination of terms

$$f_{n-1} = \sum_{\vec{\sigma},\tau,k} \frac{\mathrm{H}(x_n;\vec{\sigma})}{(x_n-\tau)^k}\,\lambda_{\vec{\sigma},\tau,k}\,, \tag{3.42}$$

where the words $\vec{\sigma}$ and poles τ depend on the remaining variables but not on x_n, while $\lambda_{\vec{\sigma},\tau,k}$ are integration constants (which can themselves involve hyperlogarithms, but in the other integration variables and parameters). This representation is achieved through a transformation that exploits the shuffle algebra and path-composition properties of iterated integrals (see, e.g., [77]). These give rise to the *shuffle product*, which, for hyperlogarithms, states that

$$\mathrm{H}(z;u)\times\mathrm{H}(z;v) = \sum_{w\in u\sqcup\!\sqcup v} \mathrm{H}(z;w)\,, \tag{3.43}$$

where $u\sqcup\!\sqcup v$ denotes the sum over all "interweavings" of the original sequence (i.e., all possible ways of merging the two ordered sequences of words u and v into a single sequence while preserving the internal order of u and v respectively; see e.g., (B.4)). This identity is used repeatedly to separate the x_n-dependent part of each word from the integration constants.

In the second step, we construct an antiderivative F satisfying $\partial_{x_n}F = f_{n-1}$ by "integrating back" the differential relation (3.41).

Step 2: Antiderivative construction. We construct an antiderivative F satisfying $\partial_{x_n}F = f_{n-1}$ by processing each term in the sum (3.42). For a given term, the

rational (in x_n) coefficient $c(x_n) \equiv \frac{\lambda_{\vec{\sigma},\tau,k}}{(x_n-\tau)^k}$ is first decomposed via partial fractions:

$$c(x_n) = p(x_n) + \sum_{i,j} \frac{c_{ij}}{(x_n - a_i)^j}\,, \tag{3.44}$$

where $p(x_n)$ is a polynomial in x_n, the a_i are the (possibly spurious, meaning it cancels in the sum) pole locations, which may depend on other variables. *Crucially*, the c_{ij} are constants in x_n. This last property is guaranteed by linear reducibility: all polynomial factors in the denominator decompose into linear terms $(x_n - a_i)$, and the standard partial fraction theorem then yields constant numerators. We handle the polynomial and pole parts separately.

For the polynomial part, let $P(x_n) \equiv \int_0^{x_n} p(t)\,\mathrm{d}t$. Integration-by-parts using (3.41) gives the antiderivative

$$\int p(x_n)\,\mathrm{H}(x_n;\vec{\sigma})\,\mathrm{d}x_n = P(x_n)\,\mathrm{H}(x_n;\vec{\sigma}) - \int \frac{P(x_n)}{x_n - \sigma_1}\,\mathrm{H}(x_n;\sigma_2,\ldots)\,\mathrm{d}x_n\,. \tag{3.45}$$

The first term contributes directly to the antiderivative; the correction term has a rational prefactor $P(x_n)/(x_n-\sigma_1)$ which is decomposed again via partial fractions, continuing the recursion. The recursion terminates when the weight reaches zero, and the hyperlogarithm in (3.45) becomes $\mathrm{H}(x_n;\varnothing) = 1$. Indeed, its derivative vanishes

$$\frac{\partial}{\partial x_n}\mathrm{H}(x_n;\varnothing) = \frac{\partial}{\partial x_n}(1) = 0\,. \tag{3.46}$$

and, consequently, no further integration-by-parts corrections are generated and we are left with rational integrands. Assuming linear reducibility, partial fractions allows to decompose these leftover integrands according to (3.44), which integrates directly since:

- ⋄ polynomials integrate to polynomials,
- ⋄ higher-order poles ($j > 1$) integrate to rational functions, via

$$\int \frac{c}{(x_n - a)^j}\,\mathrm{d}x_n = \frac{c}{(1-j)(x_n - a)^{j-1}}\,, \tag{3.47}$$

- ⋄ simple poles ($j = 1$) integrate to logarithms, via

$$\int \frac{c}{x_n - a}\,\mathrm{d}x_n = c\,\mathrm{H}(x_n;a)\,. \tag{3.48}$$

This is all we have to do for the polynomial contribution to (3.42) from (3.44).

Now we consider the pole part of (3.44). For simple poles ($j = 1$), inverting (3.41) gives directly

$$\int \frac{c}{x_n - a} \, \mathrm{H}(x_n; \vec{\sigma}) \, \mathrm{d}x_n = c \, \mathrm{H}(x_n; a, \vec{\sigma}) \,, \tag{3.49}$$

and it is clear that the pole location a just becomes a new letter prepended to the word. For higher-order poles ($j > 1$), direct integration yields a rational function which generates an integration-by-parts correction on the right-hand side:

$$\begin{aligned} \int \frac{c}{(x_n - \tau)^j} \, \mathrm{H}(x_n; \vec{\sigma}) \, \mathrm{d}x_n = & \frac{c}{(1-j)(x_n - \tau)^{j-1}} \, \mathrm{H}(x_n; \vec{\sigma}) \\ & - \int \frac{c}{(1-j)(x_n - \tau)^{j-1}} \, \frac{\mathrm{H}(x_n; \sigma_2, \ldots)}{x_n - \sigma_1} \, \mathrm{d}x_n \,. \end{aligned} \tag{3.50}$$

This correction term, after yet another partial fraction, involves poles of order at most $j - 1$ times a hyperlogarithm of lower weight. The recursion thus terminates in the exact same fashion as it did for the polynomial part and the antiderivative F for the original integral is thus obtained in terms of hyperlogarithms.

In the third and final step, we evaluate the antiderivative we found at the integration endpoints. The definite integral is obtained as

$$f_n \equiv \int_0^\infty f_{n-1} \, \mathrm{d}x_n = \lim_{x_n \to \infty} F(x_n) - \lim_{x_n \to 0} F(x_n) \,. \tag{3.51}$$

Both limits require careful treatment, as hyperlogarithms generically diverge when letters coincide with integration boundaries, which we discuss next.

3.3.3 Regularization at the endpoints

The prototypical divergence in hyperlogarithms arises from

$$\mathrm{H}(z; \underbrace{0, \ldots, 0}_{k}) = \frac{\log(z)^k}{k!} \,, \tag{3.52}$$

which diverges as $z \to 0$. Shuffle regularization extracts finite values by exploiting (B.5) to isolate divergent contributions. For instance, the identity

$$\mathrm{H}(z; a) \cdot \log(z) = \mathrm{H}(z; a, 0) + \mathrm{H}(z; 0, a) \,, \tag{3.53}$$

expresses the divergent integral $\mathrm{H}(z;a,0)$ as the difference of $\mathrm{H}(z;a)\cdot\log(z)$ and the convergent term $\mathrm{H}(z;0,a)$. Regularization then discards pure-logarithm divergences while preserving all algebraic relations.

In practice, three regularizations will arise naturally. The regularization Reg_0 handles trailing zeros in (3.52), which produce divergences as the argument approaches the lower integration bound. The regularization Reg_σ handles leading letters equal to the upper limit σ, which produce divergences as the argument approaches that limit. For instance,

$$\mathrm{H}(z;\sigma) = \int_0^z \frac{\mathrm{d}t}{t-\sigma} = -\log(\sigma - z) \to +\infty \quad \text{as } z \to \sigma\,. \tag{3.54}$$

Finally, for integrals from 0 to ∞, the regularization Reg_∞ handles constant trailing of $-\lambda < 0$. At large t, the kernel $\frac{\mathrm{d}t}{t-(-\lambda)} \sim \frac{\mathrm{d}t}{t}$ has a pole at infinity, so letters $-\lambda$ accumulate logarithmic divergences from the innermost integrations analogously to trailing zeros at the origin:

$$\mathrm{H}(\infty;\underbrace{-\lambda,\ldots,-\lambda}_{k}) = \int_0^\infty \frac{\mathrm{d}t_1}{t_1+\lambda}\cdots\int_0^{t_{k-1}} \frac{\mathrm{d}t_k}{t_k+\lambda} \sim \frac{\log(z)^k}{k!}\Big|_{z\to\infty} \to +\infty\,. \tag{3.55}$$

We can thus define the *regulated periods* as the fully regularized value that combines both boundary contributions:

$$\texttt{zeroInfPeriod}(w) \equiv \mathrm{Reg}_\infty\,\mathrm{Reg}_0\,\mathrm{H}(\infty;w)\,. \tag{3.56}$$

A crucial consistency requirement is that logarithmic divergences from the two boundaries must cancel for the integral to be well-defined.

A comment on analytic continuation. When positive letters $\lambda > 0$ lie on the integration path $[0,\infty)$, the contour must be deformed around the corresponding singularities. The choice of deformation, i.e., passing above or below each pole, can be tracked by the "delta-symbol" $\delta[\sigma] = \pm 1$, representing the sign of $\mathrm{Im}(\sigma + i\varepsilon)$. Each contour deformation contributes terms proportional to $\pm i\pi$, which combine to produce the correct analytic continuation. See [5] for more details. As a practical matter, one can fix the correct sign by matching the analytic formula with numerical evaluation.

3.3.4 A worked example

To illustrate the full procedure, consider

$$f_2 = \int_0^1 \int_0^1 \frac{\mathrm{d}x_1\,\mathrm{d}x_2}{1 - x_1 + x_1 x_2^2}\,, \tag{3.57}$$

integrated in the order (x_1, x_2) (the opposite order fails; see Sec. 3.4). The Möbius map $x_1 = x_3/(1+x_3)$ transforms the x_1-integral to $[0,\infty)$, giving $1-x_1 = 1/(1+x_3)$, $x_1 x_2^2 = x_3 x_2^2/(1+x_3)$, and hence

$$f_0 = \frac{1}{(1+x_3)(1+x_3 x_2^2)}\,. \tag{3.58}$$

Partial fractions in x_3 yield two simple poles:

$$\frac{1}{(1+x_3)(1+x_3 x_2^2)} = \frac{1/(1-x_2^2)}{x_3+1} + \frac{1/(x_2^2-1)}{x_3+1/x_2^2}\,. \tag{3.59}$$

Since the integrand is a pure rational function ($\mathrm{H}(x_3;\varnothing) = 1$, weight zero), the simple-pole rule (3.49) gives the antiderivative directly:

$$F(x_3) = \frac{1}{1-x_2^2}\,\mathrm{H}(x_3;-1) \;+\; \frac{1}{x_2^2-1}\,\mathrm{H}\!\left(x_3;-\frac{1}{x_2^2}\right)\,. \tag{3.60}$$

We now evaluate at the boundaries. At $x_3 = 0$ both hyperlogarithms vanish (no trailing zeros in the words). At $x_3 \to \infty$, each $\mathrm{H}(x_3;\sigma)$ with $\sigma < 0$ grows as $\log x_3 + \log|\sigma|$, but the leading $\log x_3$ divergences cancel because the two coefficients sum to zero: $\frac{1}{1-x_2^2} + \frac{1}{x_2^2-1} = 0$. The surviving finite parts give

$$f_1 = \frac{\log(x_2^2)}{x_2^2-1}\,. \tag{3.61}$$

For the second variable we integrate f_1 over $x_2 \in [0,1]$. Writing $\log(x_2^2) = 2\,\mathrm{H}(x_2;0)$ and decomposing $\frac{1}{x_2^2-1} = \frac{1}{2(x_2-1)} - \frac{1}{2(x_2+1)}$, the simple-pole rule gives the antiderivative

$$G(x_2) = \mathrm{H}(x_2;1,0) - \mathrm{H}(x_2;-1,0)\,. \tag{3.62}$$

At $x_2 = 0$, the trailing zeros produce a $\log(x_2)$ divergence. The shuffle identity $\mathrm{H}(x_2;\sigma)\cdot\mathrm{H}(x_2;0) = \mathrm{H}(x_2;\sigma,0) + \mathrm{H}(x_2;0,\sigma)$ gives $\mathrm{H}(x_2;\sigma,0) = \mathrm{H}(x_2;\sigma)\cdot\log x_2 - \mathrm{H}(x_2;0,\sigma)$. Then the regularization prescription discussed above discards the $\log x_2$ term, leaving $\mathrm{Reg}_0\,\mathrm{H}(x_2;\sigma,0) = -\mathrm{H}(x_2;0,\sigma)$ for $\sigma = \pm 1$. Since

$\mathrm{H}(x_2;0,\sigma) = -\mathrm{Li}_2(x_2/\sigma)$ and $\mathrm{Li}_2(0) = 0$, we get $G(0) = 0$.

At $x_2 = 1$, we apply Reg_0 (same trailing-zero identity as above) and then Reg_1 (which would strip leading letters equal to the upper limit 1, but neither $[0,1]$ nor $[0,-1]$ starts with 1, so Reg_1 acts trivially). Using $\mathrm{H}(1;0,\sigma) = -\mathrm{Li}_2(1/\sigma)$, we get

$$\mathrm{Reg}_1\,\mathrm{Reg}_0\,\mathrm{H}(1;1,0) = -\mathrm{H}(1;0,1) = \mathrm{Li}_2(1) = \zeta(2)\,, \tag{3.63a}$$

$$\mathrm{Reg}_1\,\mathrm{Reg}_0\,\mathrm{H}(1;-1,0) = -\mathrm{H}(1;0,-1) = \mathrm{Li}_2(-1) = -\frac{\pi^2}{12}\,. \tag{3.63b}$$

Combining these results, we obtain

$$f_2 = G(1) - G(0) = \frac{\pi^2}{4}\,, \tag{3.64}$$

which is correct. We discuss next what would have happened if the integration order was taken be (x_2, x_1) instead of (x_1, x_2).

3.4 Integration order and linear reducibility

The choice of the integration order plays a central role in evaluating hyperlogarithms, as it drastically affects the complexity of intermediate steps, or even whether the procedure explained in Sec. 3.3 terminates at all. For these reasons, the choice of integration order deserves separate attention.

To see this concretely, return to the integral (3.57) from Sec. 3.3.4. In the order (x_1, x_2), the polynomial $1 - x_1 + x_1 x_2^2$ is linear in x_1. After integrating x_1, the resulting letter $x_2^2 - 1 = (x_2 - 1)(x_2 + 1)$ factors linearly in x_2, and the algorithm runs to completion, yielding $\pi^2/4$.

In the opposite order (x_2, x_1), the polynomial $1 - x_1 + x_1 x_2^2$ is quadratic in x_2, and one would find the antiderivative

$$\frac{1}{1 - x_1 + x_1 x_2^2} = \frac{\partial}{\partial x_2}\left[\frac{1}{\sqrt{x_1(1-x_1)}}\arctan\left(\frac{\sqrt{x_1}\,x_2}{\sqrt{1-x_1}}\right)\right]\,. \tag{3.65}$$

The arctan function cannot be expressed as a hyperlogarithm with rational arguments, since $\arctan(z) = \frac{i}{2}[\log(1-iz) - \log(1+iz)]$ and $z = \sqrt{x_1}\,x_2/\sqrt{1-x_1}$ involves square roots. The integral is not linearly reducible in this order in the x_1, x_2 variables.

3.4.1 Fubini reduction

The question of finding linearly-reducible (LR) orders amounts to checking if at all steps of the integration, the integrand polynomials are linear in the current integration variable. The key observation is that one does not need to perform full integration to answer this question. This is because for the purposes of finding linearly-reducible orders, we only care about the singularity structure of the integrand, not the form of the integrand itself.

To this effect, we employ the strategy used in SOFIA, which uses the Fubini reduction algorithm [4, 5] to analyze singularities of integrals, see [40, Sec. 2.2]. In the first step, we use dynamical programming to find the set of singularities for every possible integration order. In the second step, we crawl the space of orders discarding those that include at least one non-linear polynomial. At the same time, we score the orders according to the metric described below.

3.4.2 Ordering orders

Once we have determined all the LR orders, we have to order them according to which one is expected to yield the most efficient integration. For this purpose, we assign scores to every LR order, which are meant to be a proxy for how complicated intermediate expressions it is expected to produce.

The intuition is that those LR orders that have large discriminants will also produce large integrands. Hence, we score each LR order by the total size (measured as the number of `Mathematica` expression leaves using `LeafCount`) of its polynomials. To corroborate this intuition, we collected a representative list of 34 example Feynman integrals. For each individual integrand term, we integrated it according to a randomly chosen LR order. We recorded the score associated with the LR orders and the eventual integration times using `HyperIntica`. The total number of data points is 10 020. The results are collected in Fig. 4. Given that the integration times and scores span multiple orders of magnitude, it is more convenient to talk about logarithms of both quantities.

The results in Fig. 4 show a moderately good correlation ($R^2 \approx 0.55$) with a power-law fit between the score S_{order} and integration time T_{order} in seconds, for any given LR order. For the mean and variance, we obtain

$$\mathbb{E}[\log T_{\text{order}}] \approx \alpha \log S_{\text{order}} + \beta \quad \text{with} \quad \alpha \approx 1.15\,, \quad \beta \approx -9.45\,, \tag{3.66a}$$

$$\mathbb{V}[\log T_{\text{order}}] \approx \sigma^2 \quad \text{with} \quad \sigma \approx 2.92\,. \tag{3.66b}$$

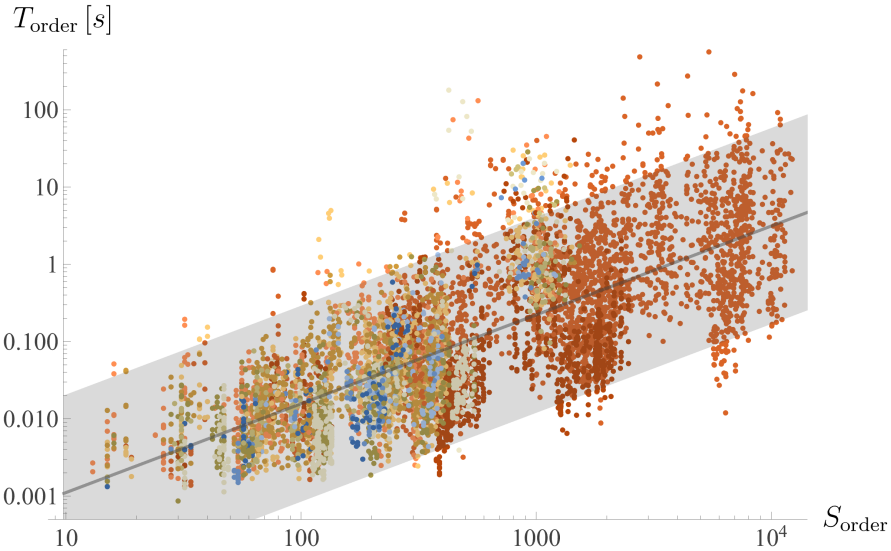

Figure 4: Correlation between the score S_{order} given to a given LR order and its resulting timing T_{order} in seconds. Different colors represent data points associated with different Feynman integrals. The straight line is the power-law fit and the shaded areas denote the 95% confidence intervals.

In particular, we find that the variance is approximately constant, which means that T_{order} follows an approximately log-normal distribution.

If we could freely choose an LR order for each integrand term, we would simply pick the one with the smallest score. However, the algorithm described in Sec. 3.2 forces us to pick the same LR order for every term belonging to the same face F of the polytope. Hence, we are tasked with a problem: how to choose a single LR order that minimizes the total expected integration time for a number of integrands?

Let us denote with $S^{(i)}_{\text{order}}$ and $T^{(i)}_{\text{order}}$, respectively, the score and the timing of the integrand $i \in F$ using a fixed order. We construct the following mean-variance cost function:

$$\mathcal{L}_{F,\text{order}} = \mathbb{E}\left[\sum_{i\in F} T^{(i)}_{\text{order}}\right] + \lambda\sqrt{\mathbb{V}\left[\sum_{i\in F} T^{(i)}_{\text{order}}\right]} \tag{3.67a}$$

$$= \mathrm{e}^{\beta+\sigma^2/2}\left[\sum_{i\in F}(S^{(i)}_{\text{order}})^{\alpha} + \lambda\sqrt{\mathrm{e}^{\sigma^2}-1}\sqrt{\sum_{i\in F}(S^{(i)}_{\text{order}})^{2\alpha}}\right], \tag{3.67b}$$

where we optimistically assumed that integration times are mutually independent.

Here, λ is the risk-aversion factor. We set $\lambda = 1.645$ to ensure 95% certainty.

In our setup, we are dominated by the variance since $\lambda\sqrt{e^{\sigma^2} - 1} \approx 117$. This should be clear from Fig. 4, since the wide error bars mean we can at best predict T_{order} within two orders of magnitude. Hence, in practice, we can essentially ignore the first term in (3.67b) and instead use

$$\tilde{\mathcal{L}}_{F,\text{order}} = \sum_{i \in F} (S^{(i)}_{\text{order}})^{2\alpha} \,, \tag{3.68}$$

where we dropped the irrelevant constants and the square root which is strictly monotonic. This is the cost function we use in the code to determine the optimal LR order. Empirically, we found that it is good at minimizing the computational effort and penalizing outliers that would otherwise stall the integration.

4 Detailed examples

In this section, we discuss in detail a representative subset of the examples available in the ancillary notebook `PaperChecks.wl`. The purpose is to illustrate both the computational capabilities and the high level of automation that `SubTropica` offers for Feynman integrals, as well as its applicability to problems beyond this context.

4.1 Scalar Feynman integrals

As the first two simple examples, we consider the massless triangle and triangle-box integral in $\mathrm{D} = 4 - 2\varepsilon$ dimensions with three distinct external masses M_1, M_2, M_3 (see Fig. 5). As the final example of this subsection, a cutting-edge case is also discussed (see Fig. 1).

4.1.1 A one-loop box with algebraic letters (Fig. 5 (left))

As a first example, consider the one-mass box with equal internal and external masses:

```
diag = {{{{1,2},m},{{2,3},0},{{3,4},m},{{1,4},0}},
        {{1,m},{2,m},{3,m},{4,m}}};
```

With the default `"FindRoots" -> False`, `STIntegrate` aborts because the integrand contains irreducible quadratic polynomials that obstruct linear reducibility. In this example, this obstruction can be resolved by enabling `"FindRoots" ->`

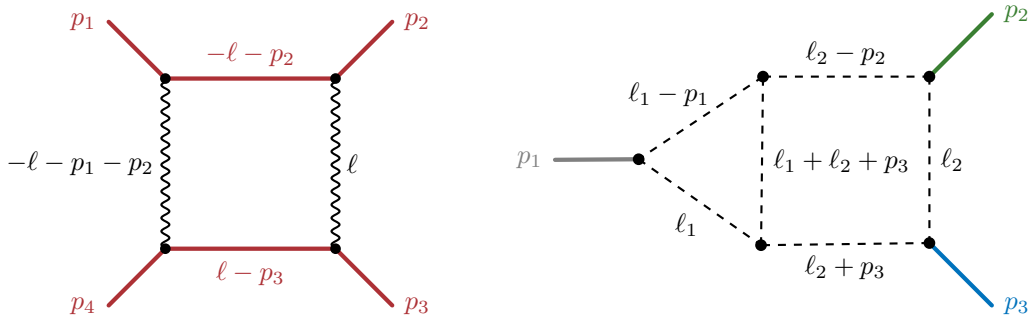

Figure 5: Left: The $e^-e^- \to e^-e^-$ (Møller) scattering one-loop box. Right: a two-loop triangle-box topology with three distinct external masses M_1, M_2, M_3. Wavy and dashed lines are massless.

`True`, which instructs `STFasterFubini` to factor these quadratics into linear factors by introducing auxiliary square-root variables, restoring linear reducibility:

```
result = STIntegrate[diag, "FindRoots" -> True]
```

The integration completes in about one second and returns

$$-\frac{\log(-W_-) - \log(-W_+)}{\mathtt{mm}\,\mathtt{s12}\,(W_- - W_+)}\left(\frac{1}{\varepsilon} - \log(-s_{12})\right) + \mathcal{O}(\varepsilon)\,, \tag{4.1}$$

where $W_\pm = (-2\,\mathtt{mm} + \mathtt{s23} \pm \sqrt{(\mathtt{s23} - 4\,\mathtt{mm})\,\mathtt{s23}})\,/\,(2\,\mathtt{mm})$ are the roots of the quadratic polynomial that obstructed linear reducibility. The result is automatically verified numerically using `STVerify`:

```
STVerify[diag, result /. GetAlgebraicBackSubRules[]]
(* Out: "pass" -> True *)
```

where `GetAlgebraicBackSubRules` replaces the expressions for $W_\pm$ in terms of m^2 and s_{23}.

4.1.2 A two-loop triangle-box integral (Fig. 5 (right)).

As a two-loop example, consider the massless triangle-box topology with three distinct external masses parametrized via the rationalizing variables z, $\bar{z}$:

```
answer = STIntegrate[
    {{{{1,4},0},{{1,5},0},{{2,5},0},{{2,3},0},{{3,4},0},{{4,5},0}},
    {{1,M1},{2,M2},{3,M3}}},
```

```
    "SetProblemID"  -> "TriangleBox",
    "Substitutions" -> {MM1 -> z zb, MM2 -> (1-z)(1-zb), MM3 -> 1}]
```

The raw result at $\mathcal{O}(\varepsilon^0)$ involves weight-4 hyperlogarithms and it can be cross-checked numerically at a specific kinematic point using `STNIntegrate`:

```
STNIntegrate[diag, "Substitutions" -> {MM1 -> z zb, MM2 -> (1-z)(1-zb),
                                       MM3 -> 1} /. {z -> 1/3, zb -> 1/7}
                   , "Order"         -> 0]
```

The numerical output agrees with the analytic result evaluated at the same point. In addition, the known result for $0 < z < \bar{z} < 1$ (see, e.g., [78, Sec. 4.3]) reads

$$\frac{\frac{1}{2}\log^2(z\bar{z})\big(\mathrm{Li}_2(z)-\mathrm{Li}_2(\bar{z})\big)-3\log(z\bar{z})\big(\mathrm{Li}_3(z)-\mathrm{Li}_3(\bar{z})\big)+6\big(\mathrm{Li}_4(z)-\mathrm{Li}_4(\bar{z})\big)}{(1-z)(1-\bar{z})(z-\bar{z})}\,, \tag{4.2}$$

and one can verify that both results agree, e.g., at the symbol-level using `ConvertToSymbol`.

Having illustrated the use of the package on simple examples, we stress that the package can also be used on certain cutting-edge Feynman integrals.

4.1.3 A cutting-edge example (Fig. 1)

As a demanding benchmark, we evaluate the four-loop, nine-propagator integral

$$\int \frac{\mathrm{e}^{4\varepsilon\gamma_E}\prod_{i=1}^{4}\frac{\mathrm{d}^{\mathrm{D}}\ell_i}{i\pi^{\mathrm{D}/2}}}{\left[(\ell_2+\ell_3+p_2+p_3)^2-m^2\right](\ell_1-\ell_3)^2\,\ell_1^2\,(\ell_1+\ell_2+p_3)^2\,\ell_2^2\,(\ell_2+\ell_3-\ell_4)^2\,\ell_3^2\,(\ell_4-p_4)^2\,\ell_4^2}\,,$$

shown in Fig. 1 at the kinematic point (s_{12}, s_{23}, m^2) where a single internal edge carries squared mass $m^2 = 1$ and all remaining internal and external masses vanish:

```
diag = {{{{4,5},0},{{1,4},0},{{1,2},1},
         {{2,3},0},{{3,5},0},{{5,6},0},
         {{2,5},0},{{4,6},0},{{1,6},0}},
         {{1,0},{2,0},{3,0},{4,0}}};

STIntegrate[diag, "Dimension"      -> 4 - 2 eps,
                  "Gauge"          -> {x3 -> 1},
                  "SimplifyOutput" -> Identity]
```

The tropical subtraction produces six locally finite integrands. The timing breakdown is:

```
>> Time to set up integrands and directories:   4.7 s
>> Time to find linearly reducible orderings:   ≈ 1.2 h
>> Time to integrate:                           ≈ 36.6 h
```

The computation completed in approximately 38 hours on a cluster equipped with an Intel Xeon Gold 6326 at 2.9 GHz (32 cores) and 256 GB of RAM, with a peak memory usage of approximately 114 GB. The result starts at $\mathcal{O}(\varepsilon^{-1})$ and is expressed in terms of hyperlogarithms. The result is too large to display here, but it can be found in the `SubTropica` library.

4.2 Linearized propagators and tensor numerators

`SubTropica` is not limited to scalar Feynman integrals with quadratic propagators: it also handles *linearized* (eikonal) propagators of the form $v \cdot k$ and loop momenta dependent (tensor) numerators. As a non-trivial illustration, we consider a family of integrals arising in the calculation of soft anomalous dimensions in [79, Eq. (D.11)].

In particular, the family is defined by the integral (in $\mathrm{D} = 4 - 2\varepsilon$):

$$\int \frac{\mathrm{d}^{\mathrm{D}}k_1}{i\pi^{\mathrm{D}/2}} \frac{\mathrm{d}^{\mathrm{D}}k_2}{i\pi^{\mathrm{D}/2}} \frac{(4\mathrm{e}^{\gamma_{\mathrm{E}}})^{2\varepsilon}[v_2{\cdot}k_1]^{-r_7}\,[\tilde{\beta}{\cdot}k_2]^{-r_8}\,[v_1{\cdot}k_2]^{-r_9}}{[k_1^2]^{r_1}\,[k_2^2]^{r_2}\,[(k_1{+}k_2)^2]^{r_3}\,[v_1{\cdot}k_1{-}1]^{r_4}\,[v_2{\cdot}k_2{-}1]^{r_5}\,[-\tilde{\beta}\cdot(k_1+k_2)]^{r_6}},$$

for the single lightcone limit of the cubic "timelike web" in [79, Eq. (D.8)]. Here, v_1, v_2 are (unit) velocity vectors of timelike Wilson lines with $v_1^2 = v_2^2 = 1$, $\tilde{\beta}$ is the velocity vector for a lightlike Wilson line ($\tilde{\beta}^2 = 0$), such that the kinematic invariants can be parametrized by $v_1{\cdot}\tilde{\beta} = -y$, $v_2{\cdot}\tilde{\beta} = -1$ and $v_1{\cdot}v_2 = -\frac{1}{2}(a_{12}{+}a_{12}^{-1})$. The propagators include both standard quadratic denominators k_i^2 and eikonalized denominators of the form $v \cdot k - 1$.

In `SubTropica`, the integral is specified by

```
(* Basis of propagators and numerators *)
basisProp = {k[1]·k[1], k[2]·k[2], (k[1]+k[2])·(k[1]+k[2]),
             v[1]·k[1]-1, v[2]·k[2]-1, -β̃·(k[1]+k[2]),
             v[2]·k[1], β̃·k[2], v[1]·k[2]};

(* An optional list of powers for each element of basisProp *)
basisExp = {1,1,1,1,1,1,-1,0,0}; (* default: all 1's *)

(* An optional list of kinematic replacements *)
```

```
basisKin = {v[1]·v[1] | v[2]·v[2] -> 1,
            v[1]·β̃ -> -y, v[2]·β̃ -> -1, β̃·β̃ -> 0,
            v[1]·v[2] -> -1/2(1/a12 + a12)};

STIntegrate[basisProp,
  "Exponents"           -> basisExp,
  "Substitutions"       -> basisKin,
  "LoopMomenta"         -> {k[1],k[2]},
  "Normalization"       -> -(4 Exp[EulerGamma])^(2 eps),
  "Gauge"               -> {x5 -> 1},
  "MethodPolysAndPairs" -> "Standard"
  ]
```

As for the Feynman diagrammatic inputs considered above, `STIntegrate` applied on the propagator and numerator basis `basisProp` automatically determines the parametric representation, finds a linearly reducible integration order across all counter-term integrands, and evaluates the ε-expansion. The first two orders read

$$-\frac{1}{4\,\varepsilon^4\,y} - \frac{\pi^2(3+7y) + 4(1+y)\log^2 a_{12} + 4y\log^2 y}{8\,\varepsilon^2\,y\,(1+y)} + \dots\,, \tag{4.3}$$

where the full result through $\mathcal{O}(\varepsilon^{-1})$ is available in the ancillary file. The result is in agreement with the differential equation method of [79].[6]

4.3 Advanced examples: Beyond Feynman integrals

We now illustrate how `SubTropica` and its lower-level functions from Tab. 1 can be used to tackle integrals that fall outside the standard Feynman integral class.

4.3.1 Gravitational energy-energy correlators

Energy-energy correlators (EECs) are infrared-finite observables that measure correlations between energy deposited at two points on the celestial sphere. In four-dimensional gravity, the reference [80] recently computed EECs at one loop in $\mathcal{N} = 8$ supergravity (SG) and in pure Einstein gravity.

In particular, they consider two gravitons scattering in the center-of-mass frame with total energy $2E$, and two calorimeters placed on the celestial sphere along

[6] We thank Zehao Zhu and Einan Gardi for providing explicit results for cross-checks.

the unit vectors $\vec{n}_1$ and $\vec{n}_2$. The geometry is parameterized by

$$\underbrace{y_1 = \frac{1-\vec{n}\cdot\vec{n}_1}{2}\,, \quad y_2 = \frac{1-\vec{n}\cdot\vec{n}_2}{2}}_{\text{beam-detector geometry}}\,, \qquad \underbrace{z = \frac{1-\vec{n}_1\cdot\vec{n}_2}{2}}_{\substack{\text{opening angle between}\\ \text{the two detectors}}}\,, \tag{4.4}$$

where $\vec{n}$ is the beam axis. The EEC can be generalized to arbitrary (complex) energy weights J_1, J_2, so that each calorimeter records $\sum_i E_i^{J_k}$ rather than $\sum_i E_i$; the standard EEC corresponds to $J_1 = J_2 = 1$. These weights are related to the spin of twist-2 operators placed at null infinity [80].

At NLO in $\mathcal{N} = 8$ SG, the real-emission contribution reduces to a univariate integral over the energy fraction $x \in [0,1]$ of the particle hitting detector one, all the other degrees of freedom already integrated out (see [80, Eq. (A.14)]):

$$\mathrm{EEC}^{\mathrm{real}}_{J_1,J_2} = \frac{E^{-4-4\varepsilon}}{16(2\pi)^{5-4\varepsilon}} \int_0^1 \mathrm{d}x\, \frac{x^{J_1+1-2\varepsilon}(1-x)^{J_2+1-2\varepsilon}}{(1-zx)^{J_2+2-2\varepsilon}}\, \mathcal{M}_{2\to 3}\,, \tag{4.5}$$

where $\mathcal{M}_{2\to 3}$ is the tree-level $2 \to 3$ *squared* matrix element summed over the $\mathcal{N} = 8$ supermultiplet:

$$\mathcal{M}_{2\to 3} = \frac{8\,E^4\,\Delta(z,y_1,y_2)}{z(1-z)\,y_1\,y_2\,(1-y_1)(1-y_2)}\,\frac{(1-zx)^4}{x^2(1-x)^2\,P(x)\,Q(x)}\,. \tag{4.6}$$

Here $\Delta(z,y_1,y_2)$ is the Gram determinant of the beam and detector directions, and $P(x;z,y_1,y_2)$, $Q(x;z,y_1,y_2)$ are quadratic polynomials in x given explicitly in [80, Eq. (3.17)] and recorded in a MATHEMATICA syntax below.

Setup. As a first step, we encode the integral (4.5) as an Euler integrand for `STIntegrate`. Since there is a single integration variable, the quadratic polynomials P and Q can be factored into linear factors using the helper function `STFactorAndTrackRoots`. The symbolic roots are stored in `explicit[P]` and `explicit[Q]` and substituted back at the end:

```
(* Factor quadratic polynomials into linear factors: *)
Ppol[x_] := STFactorAndTrackRoots[
  (1-z)(1-y1)+(z+y1-2z y1-y2)(1-x)+z y1(1-x)^2, x, P]
Qpol[x_] := STFactorAndTrackRoots[
  (1-z)y1+(y2-z-y1+2z y1)(1-x)+z(1-y1)(1-x)^2, x, Q]
```

The integral is then cast into the standard Euler form via the Möbius map $x \to x/(1+x)$, which sends $[0,1] \to [0,\infty)$. The result is packaged as a quadruple

{prefactor, integrand, variables, coefficients}:

```
(* Prefactor: *)
pref = ee^(-4-4 eps)/(16(2Pi)^(5-4 eps)) (8 ee^4 Δ)/(z(1-z) y1 y2 (1-y1)(1-y2));

(* Integrand on [0,∞) after x -> x/(1+x): *)
integrand = d[x] (x^(J1+1-2 eps)(1-x)^(J2+1-2 eps))/((1-z x)^(J2+2-2 eps)) (1-z x)^4/(x^2(1-x)^2 Ppol[x] Qpol[x])
                /. x -> x/(1+x) // Applyd[#, {x}] //. d[x_] :> 1 &;
(* Applyd computes the Jacobian of the substitution on d[x] = dx. *)

(* Variables and coefficients: *)
xvars  = {x};
coeffs = {ee, Δ, z, y1, y2};
```

Tropical analysis. For positive integer energy weights, we expect no divergences in the region $0 < z < 1$, so that the dimensional regulator ε is unnecessary. Within `SubTropica`, we can verify this explicitly with `STPreAnalysis`. For example, working near the integer values of energy weights $J_1 = 1$ and $J_2 = 2$ this command (note the Jacobian `Times @@ xvars` converting to the d log convention required by `STPreAnalysis`, see Sec. 1 and App. A.6.1)

```
(* Jacobian between the d[Log[x]] and d[x] measures *)
jac = (Times @@ xvars)
(* Tropical analysis *)
STPreAnalysis[jac integrand /.{J1->1+eps, J2->2+eps}
                            /. FactorCompletely2[a_, b_, c_] :> a,
              xvars, coeffs]
```

returns `trops -> {}`, confirming that no divergences are present. In contrast, setting, e.g., $J_1 = 0$ produces a non-empty `trops` list with entries proportional to ε, signaling a divergence regulated by the dimensional regulator alone. (A more severe situation, where an additional regulator beyond ε is required, is encountered in the small-x example discussed below.)

Expansion and integration. Since no tropical subtraction is needed around $J_1 = 1$ and $J_2 = 2$ (and, in fact, around any strictly positive integers), we can expand the integrand directly in ε and in the small deformation parameters $\delta J_1 = J_1 - 1$, $\delta J_2 = J_2 - 2$ before integrating:

```
(* Expand around J1 = 1, J2 = 2 *)
seriesJ1J2 = Series[integrand /. {J1 → 1 - δJ1, J2 → 2 - δJ2},
                    {eps, 0, 0}, {δJ1, 0, 1}, {δJ2, 0, 1}]
```

The triple expansion is then passed to `STIntegrate` order by order in $(\varepsilon, \delta J_1, \delta J_2)$.

Result. We find that the dependence on y_1 and y_2 cancels entirely, and the result depends only on the detector-detector angle z:

$$\begin{aligned}\mathrm{EEC}^{\mathrm{real}}_{J_1,J_2}\Big|_{\substack{J_1\to 1\\ J_2\to 2}} &= \frac{E^{-4-4\varepsilon}}{16(2\pi)^{5-4\varepsilon}}\frac{8\,E^4\,\Delta(z,y_1,y_2)}{z(1-z)\,y_1\,y_2\,(1-y_1)(1-y_2)}\Big[\frac{1}{2}+\frac{3}{4}(1-J_1) \\ &+\frac{(2-J_2)(1-z)\big(z+(1-z)\log(1-z)\big)}{2z^2}+\frac{(1-J_1)(2-J_2)}{4z^2}\Big[\big(1-4z+3z^2\big)\log(1-z) \\ &+z\big(3-3z-2z\,\zeta_2\big)+(4z-2)\,\mathrm{Li}_2(z)\Big]+\mathcal{O}\big(\delta J_i\delta J_j\big)\Big]\,,\end{aligned} \tag{4.7}$$

up to the overall prefactor in (4.5).

4.3.2 Fourier transforms in small-x physics

We first turn our attention to dimensionally regulated Fourier transforms arising in small-x physics. In the scattering of a high-energy quark-antiquark dipole off a hadronic target (e.g., a proton or a nucleus), there exists a frame in which the target appears Lorentz-contracted to a thin null sheet, so that the non-trivial QCD dynamics between the dipole and the target is confined to the $\mathrm{D} = 2 - 2\varepsilon$ dimensional Euclidean transverse plane. A particularly interesting integral arises in the calculation of real-virtual corrections [81, Eq. (B.9)] (see also [82]):

$$I = \frac{\mathrm{e}^{-2\varepsilon\gamma_E}}{\pi^{\mathrm{D}}}\int\frac{\mathrm{d}^{\mathrm{D}}\ell_1\,\mathrm{d}^{\mathrm{D}}\ell_2}{\ell_1^2(\ell_1-\ell_2)^2}\,\log\left(\frac{\ell_1^2}{\ell_2^2}\right)\,\mathrm{e}^{i(\ell_1\cdot X_1+\ell_2\cdot X_2)}\,, \tag{4.8}$$

where X_i and ℓ_i denote transverse positions (relative to the parent dipole) and momenta of the real and virtual emissions, respectively.

Physics aside, the presence of the logarithm in (4.8) makes it naively difficult to achieve a standard parametric form that `SubTropica` could try to evaluate directly. A natural solution is to introduce a new regulator q, so that the q-series of

$$F_q = \frac{\mathrm{e}^{-2\varepsilon\gamma_E}}{\pi^{\mathrm{D}}}\int\frac{\mathrm{d}^{\mathrm{D}}\ell_1\,\mathrm{d}^{\mathrm{D}}\ell_2}{(\ell_2^2)^{1-q}(\ell_1-\ell_2)^2(\ell_2^2)^q}\,\mathrm{e}^{i(\ell_1\cdot X_1+\ell_2\cdot X_2)}\,, \tag{4.9}$$

reproduces the original integral at $\mathcal{O}(q)$. After using the usual Schwinger trick to put all the momentum dependence of the integrand in a single exponential, we can perform the loop integrals with standard Gaussian integration tricks and obtain the parametric form (given $X_{ij} \equiv X_i \cdot X_j$)

$$F_q = \frac{\mathrm{e}^{-2\varepsilon\gamma_E}\,\Gamma(\mathrm{D}-2)}{\Gamma(q)\,\Gamma(1-q)} \int \frac{\mathrm{d}^3x}{\mathrm{GL}(1)}\, x_1^{-q}\, x_3^{q-1}\, (x_1x_2 + x_1x_3 + x_2x_3)^{\mathrm{D}/2-2} \times \left[(x_2+x_3)\,X_{11} + 2x_2\,X_{12} + (x_1+x_2)\,X_{22}\right]^{2-\mathrm{D}}. \tag{4.10}$$

Tropical analysis. If one naively expands the integrand (4.10) in q and passes it to `STIntegrate` (e.g., as we did earlier for J_1 and J_2 in the EEC example), the computation is aborted internally because a q-regulated divergence at the integration contour endpoint is not regulated by the (default) regulator ε. Once again, to diagnose this *a priori*, we use the lower-level function `STPreAnalysis` (see App. A.6.1). A first call,

```
(* Jacobian between the d[Log[x]] and d[x] measures *)
jac = (Times @@ xvars)

(* Tropical analysis *)
analysis = STPreAnalysis[jac integrand, xvars, coeffs]
```

returns `trops -> {eps}` with a single divergent ray: the tropical analysis only sees the ε-regulated divergence and is blind to the singularities involving q. To reveal the full singularity structure, we promote q to an ε-dependent regulator via, e.g., `q -> q eps`:

```
analysis = STPreAnalysis[jac integrand /. q -> q eps, xvars, coeffs]
```

Now `trops -> {eps, -eps(-1+q), -eps q}` with three divergent rays. Recall from (3.11), that each ray $\rho = (\rho_1, \rho_2)$ of the Newton polytope probes a boundary of the integration domain via the rescaling $x_i \to x_i\,\lambda^{-\rho_i}$ as $\lambda \to 0$. The associated `trops` entry gives the leading power of λ in the integrand under this rescaling: if it vanishes, the divergence along that ray is unregulated.

To see this concretely, we can use `STFactor` (see App. A.6.4) to inspect the λ-dependence of the integrand along each ray:[7]

[7] In contrast, the ray $\{-1,-1\}$ ($x_1 \to x_1\lambda$, $x_2 \to x_2\lambda$) makes the integrand scale as $\lambda^{1-\varepsilon-q}$, which vanishes as $\lambda \to 0$ and thus sources no divergence.

```
(* Ray {1,1}: x1 -> x1/λ, x2 -> x2/λ *)
STFactor[jac integrand/. {x1 -> x1/λ, x2 -> x2/λ}]
  (* → λ^q: regulated by q only *)

(* Ray {0,1}: x1 fixed, x2 -> x2/λ *)
STFactor[jac integrand/. {x1 -> x1, x2 -> x2/λ}]
  (* → λ^(-ε): regulated by ε *)

(* Ray {1,0}: x1 -> x1/λ, x2 fixed *)
STFactor[jac integrand/. {x1 -> x1/λ, x2 -> x2}]
  (* → λ^(-ε+q): (already) regulated by ε *)
```

Clearly, setting $q = 0$, only the entry of `trops` along ray $\{1, 1\}$ vanishes, confirming that this is the only divergence genuinely regulated by q. The other rays remain regulated by ε alone. This tells us that the ray $\{1, 1\}$ must be treated separately before expanding in q, which is what we discuss next.

Nilsson–Passare continuation. The q-regulated divergence along ray $\{1, 1\}$ is handled by the tropical continuation (see Sec. 3.2.2), applied via `STTropicalContinuation` (see App. A.6.2):

```
{{pref1, int1}, {pref2, int2}} =
    STTropicalContinuation[{{pref,jac integrand}},xvars,{{1,1}}]

(* pref1 = Exp[-2 eps EulerGamma](1 + eps) Gamma[-2 eps] Sin[π q]/(π q) *)

(* int1  = x1^(1-q) x2 (x1 + x2)(x1+x2+x1 x2)^(-2-eps)
            (x1+x2-x2 z-x2 zb+z zb+x2 z zb)^(2 eps) *)

(* pref2 = Exp[-2 eps EulerGamma]z zb Gamma[1 - 2 eps] Sin[π q]/(π q) *)
(* int2  = x1^(1-q) x2(x1+x2+x1 x2)^(-1-eps)
            (x1+x2-x2 z-x2 zb+z zb+x2 z zb)^(-1+2 eps) *)
```

The output is a list of two `{prefactor, integrand}` pairs.

The continuation extracts the q-pole along ray $\{1, 1\}$ and decomposes the original integral into a sum of terms where the singular q-dependence (including the $\Gamma(q)\Gamma(1-q)$ poles from (4.10)) is isolated in the prefactors. In both resulting integrands, the only remaining q-dependence is a factor x_1^{1-q} in the $\mathrm{d}\log$ measure, i.e., x_1^{-q} in the standard $\mathrm{d}x$ measure (see Sec. 1). One can verify that this does not introduce any new unregulated divergence by running `STPreAnalysis` on the two

integrands: all `trops` entries are nonzero at $q = 0$ (or absent altogether for the second term) with no new divergent ray detected, confirming that every divergence is regulated by ε alone.

Expansion in q and tropical subtraction in ε. With the q-pole now isolated in the prefactors, we extract the $\mathcal{O}(q)$ coefficient of each term via `SeriesCoefficient`, recovering the logarithmic integrand of the original problem (4.8) and obtaining q-independent integrands that still carry ε-divergences. These are then passed to `STExpandIntegral`, which performs the standard tropical subtraction to render them locally finite in ε:

```
(* Extract O(q) coefficient of each term *)
int1 = SeriesCoefficient[pref1 int1, {q, 0, 1}];
int2 = SeriesCoefficient[pref2 int2, {q, 0, 1}];

(* Tropical subtraction in eps *)
serI1 = STExpandIntegral[int1, xvars, coeffs]
serI2 = STExpandIntegral[int2, xvars, coeffs]
```

The locally finite integrands in `serI1` and `serI2` are then integrated order-by-order in ε using `HyperIntica` (see `PaperChecks.wl` for details).

The raw result is a lengthy expression of hyperlogarithms. After simplification via symbol and constant matching (see `PaperChecks.wl` for the detailed procedure), the result takes the compact form (in the region where $(z - \overline{z})^2 < 0$ enforcing that $\overline{z} = z^*$ or, equivalently the Cauchy–Schwarz inequality at the level of the transverse distances squared $X_{12}^4 \leqslant X_{11}^2 X_{22}^2$ in (4.8))

$$\begin{aligned} I = \frac{1}{2\varepsilon^3} + \frac{\zeta_2}{2\varepsilon} + 4\,\mathrm{Re}[\mathrm{Li}_3(1-z)] + 4\,\mathrm{Re}[\log(1-z)]\,\mathrm{Re}[\mathrm{Li}_2(z)] \\ + 4\,\mathrm{Re}[\log(1-z)](\mathrm{Re}[\log(1-z)\log z] - \zeta_2) + \frac{\zeta_3}{3} + \mathcal{O}(\varepsilon)\,. \end{aligned} \tag{4.11}$$

We checked that this result agrees perfectly with the one computed using differential equations in [81].

4.4 A standalone `HyperIntica` example

Finally, we demonstrate `HyperIntica` as a standalone integration engine, independent of `SubTropica`'s Feynman integral pipeline. Consider the five-variable

integral with a cubic logarithmic numerator:[8]

```
num = t2^2 t3^4 t4^2 t5
  (3 Log[t1] + 5 Log[t2] - 87 Log[t3] - 17 Log[t1+t2+t3] - 12 Log[t4]
   + 19 Log[t5] + 9 Log[1+t5] + 24 Log[(1+t1+t2) t3 + t2 t4 t5]
   + 50 Log[(1+t1+t2) t3+t2 t4 (1+t5)] - 3 Log[t1 t3+(t2+t3)(t3+t4 t5)])^3;

den = 6 (t1+t2+t3)^2 (1+t5)((1+t1+t2) t3 + t2 t4 t5)^2
        ((1+t1+t2) t3 + t2 t4 (1+t5))^2(t1 t3 + (t2+t3)(t3 + t4 t5))^2;

integrand = num/den;
```

This integrand is a hyperlogarithm-valued rational function in the five variables $t_1, \ldots, t_5$, and its integral over $[0, \infty)^5$ is a period (i.e., a number). Linear reducibility requires that the set of irreducible polynomial factors remains linear in the next integration variable at every step. Here the relevant polynomials are the non-monomial arguments of the logarithms and the irreducible factors of the denominator. Since these coincide, the polynomial set is

```
polys = {t1+t2+t3, 1+t5, (1+t1+t2) t3 + t2 t4 t5,
         (1+t1+t2) t3 + t2 t4 (1+t5), t1 t3 + (t2+t3)(t3 + t4 t5)};
```

Passing these (together with the variables) to `STFasterFubini` (App. A.7) finds a linearly reducible order:

```
STFasterFubini[{Join[polys, {t1,t2,t3,t4,t5}]},
                 {t1,t2,t3,t4,t5}]
(* Out: {t4, t5, t1, t3, t2} *)
```

The integral is then evaluated by passing it directly to `HyperIntica`:

```
HyperIntica[integrand, {t4, t5, t1, t3, t2}, "Monitor" -> True]
```

It is worth mentioning that for this integral, the built-in heuristic in `STFasterFubini` (see Sec. 3.4) does *not* return the fastest integration order, which turns out to be `{t4, t5, t1, t2, t3}` (note the swapped last two variables). This illustrates the difficulty of predicting the optimal order from compatibility graph reduction and the heuristic implemented in `SubTropica` alone; we return to this point in Sec. 5.

[8] We would like to thank Vladimir A. Smirnov for suggesting this example to us.

For both orders, the computation processes each variable sequentially (approximately 6300 seconds total on a laptop for the optimal order) and returns the same linear combination of multiple zeta values:

$$\begin{aligned}\int_{[0,\infty)^5} \texttt{integrand}\, \mathrm{d}^5 t \;&= \tfrac{744301}{8} + \tfrac{67803}{2}\,\zeta_2 - \tfrac{1375959}{20}\,\zeta_2^2 + \tfrac{1234511}{15}\,\zeta_2^3 - \tfrac{3889163}{24}\,\zeta_3 \\ &\quad - \tfrac{43941}{2}\,\zeta_2\,\zeta_3 + \tfrac{584512}{5}\,\zeta_2^2\,\zeta_3 + 3344\,\zeta_3^2 - \tfrac{109324}{3}\,\zeta_5 \\ &\quad - 27354\,\zeta_2\,\zeta_5 - \tfrac{1142764}{3}\,\zeta_7\,, \end{aligned} \tag{4.12}$$

which was cross-verified with `HyperInt`.

5 Conclusion

We have presented `SubTropica`, a MATHEMATICA package for the automatic evaluation of linearly reducible Euler integrals via tropical subtractions. The package combines the tropical subtraction algorithm of [1] with `HyperIntica`, a self-contained reimplementation of [7], providing an end-to-end workflow entirely within MATHEMATICA. As demonstrated through examples ranging from multi-loop Feynman integrals to phase-space integrals, Fourier transforms in high-energy QCD, and energy correlators, the package achieves a high level of automation while remaining accessible through a graphical interface. We have also released the `SubTropica` library, a repository of Feynman integrals computed in the literature together with their results.

Several directions for improvement and extension are worth mentioning.

Extending the domain of physical applications. While a large number of physical computations are naturally formulated in terms of Euler integrals already, we expect that the class of applications can be enlarged further. For instance, phase-space integrals have, in some cases, been rewritten in this form, see, e.g., [74, 83, 84]. Other examples include correlation functions for large-scale structure in cosmology, which can often be brought into Euler integral form [85] (see also [86]). With or without `SubTropica`, being able to achieve this systematically would unlock a whole new class of computational tools. We leave it as a challenge to the community.

Improving tropical subtraction schemes. Recall from Sec. 3.2.2 that one of the current bottlenecks is the need for Nilsson–Passare continuation whenever the

integrand does not satisfy the geometric property. Circumventing this limitation would yield enormous speedups for certain classes of integrals, and hence should be explored more thoroughly. As an encouraging example, Hillman's tropical subtraction formula [2] achieves this objective for integrals with UV divergences. A concrete goal would be to extend it to IR divergences as well.

`C++` backend. Another bottleneck, specific to MATHEMATICA, is its slow implementation of polynomial operations such as `Series`, `Apart`, `Together`, and `Factor`. The clear path forward is to introduce a `C++` backend implementing these functions. We plan to pursue this in the immediate future.

Parallelization. For every face of the Newton polytope, `HyperIntica` is launched in parallel on multiple integrals. In practice, we find that a small subset of integrals consumes most of the compute. A better strategy would weight integrals by a predicted-complexity estimate to balance the load. Additionally, it would be desirable to explore possible parallelization of the standalone package `HyperIntica`.

Extending the class of integrals. The requirement of linear reducibility, while satisfied by a large class of physically relevant integrals, is a genuine limitation. For example, transverse $\mathrm{d}z \wedge \mathrm{d}\bar{z}$ integrals in, e.g., high-energy QCD [87] would require `HyperIntica` to account systematically for holomorphic anomalies. Moreover, although `SubTropica` deals with some integrals involving algebraic (e.g., square-root) letters, automatic detection of such features is not yet systematic. On the other hand, algorithmic methods of rationalization do exist, see, e.g., [88]. We have implemented prototype AI-assisted strategies to address this problem, which will be featured in a future release of `SubTropica`.

Finding linear orders. Even though we have found simple-to-compute heuristic metrics that correlate reasonably well with the eventual integration time (see Sec. 3.4.2), the predictor still has an enormous variance, spanning three orders of magnitude. In practice, this means a single integral can occasionally dominate the wall-clock budget, clogging the pipeline. More reliable ways of finding linearly-reducible orders, perhaps AI-assisted, therefore remain a priority.

Extensions beyond hyperlogarithms? The tropical subtraction scheme is completely general and it can be applied to any Euler integral, regardless of whether it is multi-polylogarithmic or not. From this point of view, it can be

used in conjunction with any future direct-integration software, e.g., for elliptic iterated integrals. We look forward to seeing progress in this direction.

Expansion by regions. A natural extension of `SubTropica` would be to implement the expansion by regions (see, e.g., [60]), which is already formulated in terms of polyhedral geometry. We intend to add this feature in a future release.

Crown problems. The issue of non-positive integrands mentioned in Sec. 3.1 is not only of practical concern, but also tied to a deep problem of defining perturbative quantum field theory in the first place. It is not entirely clear that in such cases Feynman integrals are analytic, or, more precisely, boundary values of a single analytic function [63]. It is plausible that one is forced to split the integral into multiple analytic functions, each with a different kinematic $i\epsilon$ prescription. Curiously, one of the simplest examples of this problem, the crown diagram (Fig. 2), belongs to a master-integral family computed in [89, 90], suggesting that differential equations might serve as a guide for understanding the issue of non-positive Feynman integrals.

`SubTropica` as an end-to-end tool for scattering amplitudes. With a simple click-to-integrate approach, we demonstrated that one does not need to be an expert on Feynman integrals to be able to compute them. There is no reason to stop here. In the future, we plan to extend this approach to scattering amplitudes wholesale, streamlining the full pipeline from the Lagrangian to the observable. We imagine a future in which every computation in quantum field theory will be only a single `Shift+Enter` away.

Acknowledgments. We thank Luca Abu El-Haj, Wayne Ariston, Miguel Correia, Giulio Crisanti, Einan Gardi, Johannes Henn, Won Lim, Ian Moult, Audrey Pepin, Vladimir Smirnov, John Staunton, Zehao Zhu for discussions, suggesting example applications, feature requests, and beta-testing. M.G. is supported by the U.S. Department of Energy (DOE) grant No. DE-SC0011941. The work of G.S. is also supported in part by the DOE (Grant No. DE-SC0009988), further support was made possible by the Carl B. Feinberg cross-disciplinary program in innovation at the IAS. The work of G.S. is part of the PositiveWorld project funded by the European Union's Horizon 2023 research and innovation programme under the Marie Skłodowska-Curie grant agreement 101151760. Funded by the European Union (ERC, UNIVERSE PLUS, 101118787).

Views and opinions of the authors expressed are those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.

A Function reference

A.1 Detailed package configuration: `ConfigureSubTropica`

The full list of possible options supplied to `ConfigureSubTropica` is:

```
ConfigureSubTropica[
    (* required *)
    PolymakePath                -> "path/to/polymake",

    (* polynomial-arithmetic backend (recommended) *)
    FiniteFlowPath              -> "path/to/finiteflow/mathlink",
    SPQRPath                    -> "path/to/SPQR",

    (* numerical verification backends (optional) *)
    GinshPath                   -> "path/to/ginsh",
    FIESTAPath                  -> "path/to/FIESTA5",
    AMFlowPath                  -> "path/to/amflow",
    FeyntropPath                -> "path/to/feyntrop",
    PythonPath                  -> "path/to/python3",

    (* symbolic integration (optional) *)
    MaplePath                   -> "path/to/maple",
    HyperIntPath                -> "path/to/HyperInt.mpl",

    (* IBP reduction (optional, used by AMFlow) *)
    FIREPath                    -> "path/to/FIRE6",
    LiteRedPath                 -> "path/to/LiteRed",
    LiteIBPPath                 -> "path/to/finiteflow-mathtools/
       packages",

    (* tuning *)
    PolymakeConcurrencyFraction -> 0.75,
    BenchmarkNudge              -> True
];
```

The options of `ConfigureSubTropica` listed above fall into three groups. The first group consists of paths to tools already introduced in the main text: `Poly-`

makePath, GinshPath, MaplePath, PythonPath, and the pair FiniteFlowPath and SPQRPath. Two of these options deserve a brief comment. HyperIntPath points at HyperInt.mpl file itself rather than a containing directory; leaving this option empty triggers auto-discovery under ~/HyperInt, ~/hyperint, and /opt/HyperInt. PythonPath selects a specific Python *interpreter*, not a package root: the canonical use case is a virtual environment whose site-packages directory contains pySecDec.

The second group configures the numerical verification backends invoked by STVerify and by the GUI validator. In addition to FIESTAPath, which points at the directory containing FIESTA5.m, one may set AMFlowPath and FeyntropPath, which enable two further backends: AMFlow [33], based on auxiliary mass flow, and feyntrop [34], based on tropical Monte–Carlo sampling. AMFlowPath points at the root directory of AMFlow (containing AMFlow.m), while FeyntropPath points at the directory holding the compiled feyntrop binary. Because AMFlow delegates its IBP-reduction step to an external reducer, three further path options are provided: FIREPath, LiteRedPath, and LiteIBPPath, which locate FIRE6 [16], LiteRed [18], and finiteflow-mathtools [24] package, respectively. When more than one reducer is available, SubTropica dispatches automatically.

The third group consists of two runtime knobs. PolymakeConcurrencyFraction (default 0.75) caps the fraction of the available CPU cores consumed by concurrent polymake jobs, and the boolean BenchmarkNudge (default True) toggles a one-line reminder in the welcome banner suggesting a call to STBenchmark[] on first use of a fresh installation. All options discussed above are auto-persisted to the file $STConfigFile and replayed on subsequent loads, so ConfigureSubTropica need only be called once per machine, or whenever a dependency is moved or upgraded.

A.2 Complete option reference: STIntegrate

Option reference. The available options, grouped by purpose, are:

```
STIntegrate[input,
(* A: Core settings *)
"Order"                    -> Automatic,
"Dimension"                -> 4 - 2 eps,
"Normalization"            -> Automatic,
"Substitutions"            -> {},
"Representation"           -> "Schwinger",
"Integrator"               -> "HyperIntica",
(* A': Propagator input only *)
```

```
"Exponents"                  -> Automatic,
"LoopMomenta"                -> Automatic,
(* B: Gauge selection and scoring *)
"Gauge"                      -> Automatic,
"IncludeGauges"              -> All,
"Heuristic"                  -> "LeafCountLinear",
"ScanScoreInterval"          -> {1, 3},
"ScoreInParallel"            -> All,
"TimeUpperBound"             -> 10^17,
"ScoringMemoryFraction"      -> 0.5,
"MethodLR"                   -> "Lungo",
"MethodPolysAndPairs"        -> "Fast",
"FindRoots"                  -> True,
(* C: Parallelization and memory *)
"KernelsAvailable"           -> $ProcessorCount - 1,
"ClearCachesPerIntegrand"    -> False,
(* D: Output and diagnostics *)
"Verbose"                    -> False,
"ShowTimings"                -> True,
"ShowIntegrands"             -> False,
"SaveSlowestIntegrand"       -> False,
"SetProblemID"               -> Automatic,
"SimplifyOutput"             -> Simplify,
"CleanOutput"                -> False,
"ContourHandling"            -> "Abort",
(* E: Checkpointing and workflow *)
"StopAt"                     -> Automatic,
"StartAt"                    -> None,
"SelectFaces"                -> All,
"ReuseExistingResults"       -> True]
```

This list, as well as all the values each option can take can be called in a session using `STOptionValues[]`.

A: Core settings. `"Order" -> Automatic` (default) returns the result through the finite part, i.e., through $\mathcal{O}(\varepsilon^0)$. Setting `"Order" -> n` returns the result through $\mathcal{O}(\varepsilon^n)$. The required integration depth is determined automatically from the pole structure of the overall prefactor (e.g., from $\Gamma(\varepsilon)$ factors), so the user never needs to worry about how many orders `HyperIntica` must compute internally not to miss ε/ε contributions. ✧

`"Dimension"` sets the spacetime dimension, e.g. `"Dimension" -> 6 - 2*eps`.

`"Normalization" -> Automatic` includes the standard $e^{L\gamma_E\varepsilon}$ prefactor for an L-loop integral. `"Substitutions"` accepts replacement rules applied to kinematic invariants before integration, e.g. `{s12 -> 1, mm -> 0}`. `"Representation"` selects `"Schwinger"` or `"LeePomeransky"` [91], where the Schwinger representation typically yields simpler polynomial structures and thus simpler integration. `"Integrator"` selects the symbolic integration backend: `"HyperIntica"` (default) uses the built-in MATHEMATICA implementation, while `"HyperInt"` delegates to the `Maple` implementation of [7] (requires `MaplePath` and `HyperIntPath` to be set via `ConfigureSubTropica`). ✧

A′: Propagator input only. `"Exponents" -> Automatic` assigns unit exponent to every propagator by default. Setting `"Exponents" -> {`n_1`, `n_2`, . . . }` assigns the exponent n_i to the ith entry. Negative values insert the corresponding polynomial as a (possibly tensor) numerator factor. ✧

The default option `"LoopMomenta" -> Automatic` assumes that the loop momenta are labeled by `{l[1], l[2], ... }` in the propagator list. The user can set the option to e.g., `{k[1], k[2], ... }`, to specify them manually. ✧

B: Gauge selection and scoring. `SubTropica` automatically detects whether the input integrand is homogeneous in the integration variables. When it is (e.g., for integrands arising from the Schwinger representation of a Feynman graph), the code assumes an overall GL(1) redundancy and searches for the gauge choice $x_i = 1$ that yields the simplest integration (according to the scoring metric set by `"Heuristic"`, which is discussed below).

`"Gauge" -> Automatic` (default) scans all candidate gauges, scores them, and selects the best one, while `"Gauge" -> {x3 -> 1}` fixes the gauge directly. `"IncludeGauges" -> {1,2,5}` restricts the automatic search to $x_1 = 1$, $x_2 = 1$, $x_5 = 1$, whereas `All` (default) considers every Feynman parameter.

Note that if the integrand is *not* homogeneous (e.g., the Lee–Pomeransky representation of a Feynman graph), `SubTropica` detects this automatically and integrates over all parameters without gauge fixing. When necessary, this behavior can also be forced manually even on homogeneous integrands via `"Gauge" -> {}`. ✧

`"Heuristic"` controls how candidate linearly reducible integration orders are ranked. The default `"LeafCountLinear"` minimizes the sum of leaf counts of intermediate polynomials, while alternatives include `"ByteCountLinear"`, `"TermCountLinear"`, `"TotalDegreeLinear"`, and root-mean-square variants. ✧

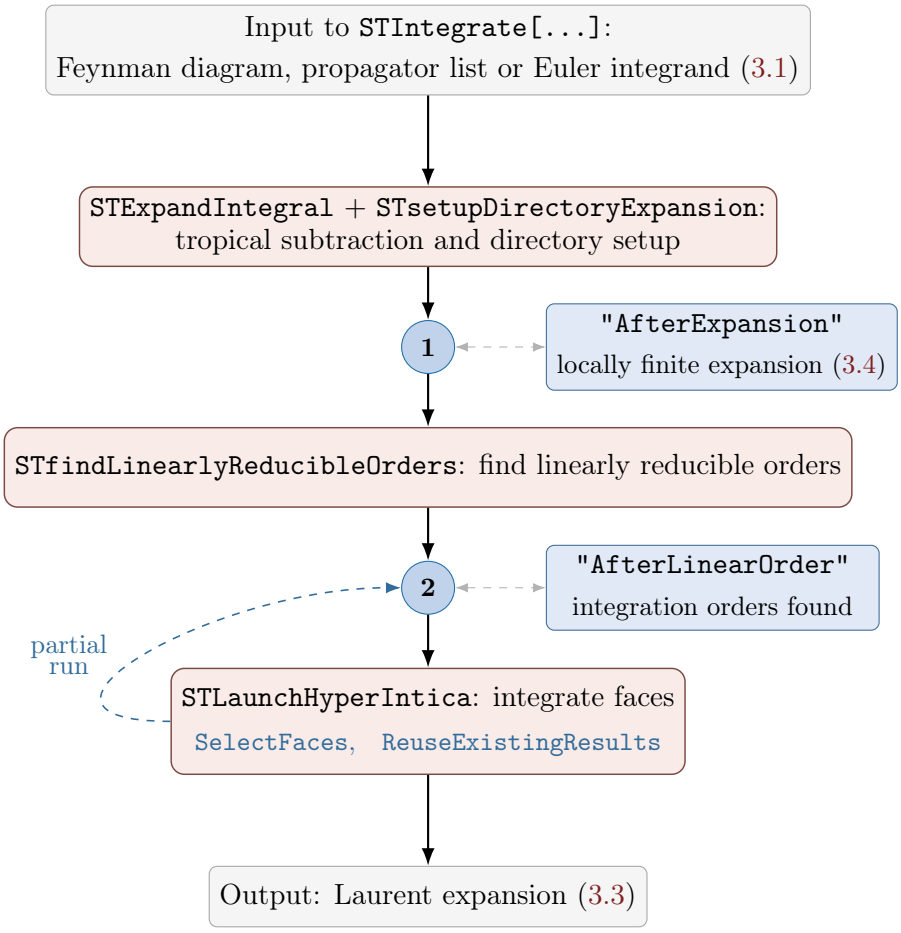

Figure 6: The algorithmic steps of `STIntegrate` described in Sec. 3. Numbered circles mark the two checkpoints where the pipeline can be paused via `"StopAt"` and resumed later via `"StartAt"`. The single-headed dashed feedback arrow indicates that the integration step can be run multiple times with different `"SelectFaces"` selections, with completed faces reused via `"ReuseExistingResults"`.

On complicated examples, gauge scoring can be memory-intensive. `SubTropica` implements options to spread the memory pressure when needed. The option `"ScanScoreInterval" -> {m,n}` restricts scoring to faces m through n, reducing cost. `"ScoreInParallel" -> k` splits candidates into k parallel chunks, while `All` (default) processes all gauges in one batch. `"ScoringMemoryFraction" -> f` ($0 < f \leqslant 1$, default 0.5) sets the fraction of available RAM budgeted for parallel scoring, controlling the number of gauges scored concurrently.

The option `"TimeUpperBound" -> T` sets a per-gauge time limit in seconds and candidates exceeding it are deprioritized. ✧

`"MethodLR"` selects the algorithm used to find linearly reducible integration orders. The default `"Lungo"` uses an optimized Fubini reduction algorithm. The alternative `"Espresso"` combines it with deduplication via proportional-polynomial intersection, which is typically faster but might lead to sub-optimal integration orders. ✧

`"MethodPolysAndPairs"` controls how the polynomial sets used for the linear reducibility check are extracted from the integrands and counter-terms. The default `"Fast"` reads the irreducible polynomial factors directly from the internal monomial-polynomial decomposition (`STtoCoeffMonPols`) of each counter-term integrand *before* expanding in ε, which avoids the cost of assembling and refactoring the summed integrand at each ε-order. The alternative `"Standard"` first sums all integrands and counter-terms at each ε-order and then extracts the polynomial set via `STpreparePolysAndPairs`, which can detect cancellations between counter-terms that reduce the polynomial set. When `"Fast"` is selected (default) and no linearly reducible order is found, `STIntegrate` automatically falls back to `"Standard"` and retries, since the reduced polynomial set relevant up to the specified `Order` may admit a linearly reducible order that the all-orders `"Fast"` set does not allow. ✧

`"FindRoots" -> True` (default) attempts to factor univariate polynomials of degree greater than one during the linear-order search by introducing auxiliary root variables (e.g., `Wm[i]`/`Wp[i]`), extending the class of linearly reducible integrals at the cost of increased complexity. Setting it to `False` restricts the search to orders that are linearly reducible without algebraic letters. ✧

C: Parallelization and memory. `"KernelsAvailable" -> n` sets the number of sub-kernels (default `$ProcessorCount` − 1). Setting `"KernelsAvailable" -> 0` or `1` runs the entire pipeline in serial on the main kernel: no sub-kernels are launched, and all `ParallelTable`/`ParallelMap` calls fall back to their serial

equivalents automatically. Sub-kernels are launched on the first `STIntegrate` call and reused across subsequent calls in the same session; it is advisable to restart kernels after large runs to release accumulated memory. ✧

`"ClearCachesPerIntegrand" -> False` (default) clears all sub-kernel memoization caches only once, after all counter-term integrands for a given diagram have been integrated, maximizing cache reuse. Setting it to `True` clears caches after each individual integrand, which reduces peak memory at the cost of losing intermediate integrand cache reuse. ✧

When `FiniteFlow` [24] and `SPQR` [54] are loaded, the potentially costly partial-fraction step inside `HyperIntica` uses finite-field reconstruction via `SPQRPolynomialQuotient`, avoiding intermediate expression swell. This introduces an independent layer of parallelism (via `FiniteFlow`'s internal `C++` threads) that is non-interfering with MATHEMATICA's sub-kernel scheduler. The weight range used by the reconstruction is controlled by `$PartialFractionsMaxWeight` (default `{1,100}`) and can be overridden per call via `PartialFractions[f, x, "MaxWeight" -> {1, 200}]`. ✧

D: Output and diagnostics. `"Verbose" -> True` enables detailed print output throughout the computation. `"ShowTimings" -> True` (default) prints timing for each stage, and `"ShowIntegrands" -> True` shows a live monitor of the integrand running on each kernel, which is useful for spotting bottlenecks in real time. ✧

`"SaveSlowestIntegrand" -> "file.m"` records the slowest integrand after the run (default `False`). ✧

`"SetProblemID" -> Automatic` generates a unique run identifier used to name intermediate directories, and can be set to any string. `"SimplifyOutput" -> f` applies `f` to the final result. Passing `Identity` skips simplification entirely. ✧

`"CleanOutput" -> True` applies `CleanZeroInf[]` to resolve any reducible remaining `zeroInfPeriod` expressions into explicit multi-zeta values before `"SimplifyOutput"` is applied. ✧

`"ContourHandling" -> "Abort"` (default) controls how the code responds when the contour direction around a singularity on the real axis cannot be determined unambiguously during the evaluation of $\mathrm{H}(\infty;\ldots)$ limits. The default `"Abort"` halts the computation with a diagnostic message identifying the problematic letter. Setting `"ContourHandling" -> "Continue"` instead leaves the offending $\mathrm{H}(\infty;\ldots)$ unevaluated and continues, which can be useful for debugging or when the user intends to resolve the contour prescription externally. ✧

E: Checkpointing and workflow. For difficult or long-running computations, the pipeline can be paused at intermediate stages and resumed later, and individual tropical faces can be targeted selectively.

`"StopAt"` interrupts the pipeline and saves an intermediate result to disk as a checkpoint under `checkpoints/<ID>.m` (where `<ID>` is the problem identifier set by `"SetProblemID"`) that can be reloaded later via `"StartAt"`. The accepted values are `"AfterBuildingIntegrand"` (returns the raw parametric integrand `{prefactor, integrand, variables, coefficients}`), `"AfterExpansion"` (returns the list of locally finite integrands from the tropical subtraction without launching `HyperIntica`), and `"AfterLinearOrder"` (additionally records the linearly reducible integration orders). The default `Automatic` runs the full pipeline without saving any checkpoint. The lower-level functions invoked at each stage, `STExpandIntegral` (at `"AfterExpansion"`) and `STFasterFubini` (at `"AfterLinearOrder"`), are documented in App. A.6.3 and A.7 respectively (see also Fig. 6). The valid values for `"StopAt"` and all other options with enumerable values can be queried interactively via the helper function `STOptionValues[]`. For instance, `STOptionValues["StopAt"]` returns the list of accepted strings. ✧

`"StartAt"` resumes the pipeline from a checkpoint previously saved by `"StopAt"`. The argument is the checkpoint identifier returned by the earlier call. ✧

`"SelectFaces" -> All` (default) passes all tropical face integrands to `HyperIntica`. Setting it to a single integer n or a list $\{n_1, n_2, \ldots\}$ restricts integration to the faces with those indices. The syntax `o -> i` selects face i at ε-order o only, and lists of such pairs are accepted as well. The pattern `Except[f]` integrates every face not matched by `f`, where `f` can be any of the above forms. Face contributions that have not yet been integrated appear as `STwrapError[...]` placeholders in the assembled output until the missing faces are completed. ✧

`"ReuseExistingResults" -> True` (default) checks, before calling `HyperIntica` on each face, whether a completed `result.m` and `successQ.m` already exist on disk. If so, the face is skipped and its stored result is read back at assembly time. Setting this to `False` forces every selected face to be (re)integrated regardless of what is on disk. This option is most useful in combination with `"SelectFaces"` and `"StartAt"`, as illustrated below. ✧

The following example illustrates a typical multi-stage workflow for a diagram "`diag`":

```
(* Stage 1: run tropical subtraction and stop before integration *)
stage1 = STIntegrate[diag, "StopAt" -> "AfterExpansion"];

(*Stage 2: additionally find linearly reducible integration orders*)
stage2 = STIntegrate[diag, "StopAt" -> "AfterLinearOrder"];

(*Stage 3: resume from any checkpoint and complete the integration*)
res = STIntegrate[diag, "StartAt" -> stage1["CheckpointID"]];
res = STIntegrate[diag, "StartAt" -> stage2["CheckpointID"]];

(* Stages can be combined: stop and resume in a single call *)
stage2 = STIntegrate[diag, "StartAt" -> stage1["CheckpointID"],
                           "StopAt" -> "AfterLinearOrder"];

(* To perform the integration from any of the stages, simply run *)
res = STIntegrate[diag, "StartAt" -> stage2["CheckpointID"]];
```

Note that checkpoints persist between different examples and across kernel restarts, since they are written to `checkpoints/<ID>.m`. ✧

Targeting specific faces. Similarly, for difficult computations it can also be useful to integrate one subset of faces at a time, for instance when a single face is slow or failing while the others complete quickly. This requires first anchoring all calls to the same problem directory via `"StopAt"`/`"StartAt"`, and then using `"SelectFaces"` to route each call to the desired subset. A typical workflow for a diagram with four faces is:

```
(* First obtain a checkpoint to anchor all subsequent calls *)
stage = STIntegrate[diag, "StopAt" -> "AfterLinearOrder"]

(* Integrate only faces 1 and 4 (all epsilon orders): *)
STIntegrate[diag, "StartAt" -> stage["CheckpointID"],
              "SelectFaces" -> {1, 4}]
(*Contributions from skipped faces are flagged by STwrapError[...]*)

(* Integrate the remaining faces reusing faces 1 and 4 on disk *)
STIntegrate[diag, "StartAt" -> stage["CheckpointID"],
              "SelectFaces" -> Except[{1, 4}],
     "ReuseExistingResults" -> True]

(* Collect the full result; all faces are now on disk *)
res = STIntegrate[diag, "StartAt" -> stage["CheckpointID"],
```

```
            "ReuseExistingResults" -> True]

(* Faces can also be selected by {epsOrder -> faceIndex} pairs *)
STIntegrate[diag, "StartAt" -> stage["CheckpointID"],
              "SelectFaces" -> {0 -> 1, 0 -> 2}]
```

The accepted `"SelectFaces"` syntax is summarized below:

- ◇ `All`: every face at every ε-order (default).
- ◇ `n` or `{n}`: face index n at all ε-orders.
- ◇ `{n1, n2, . . . }`: face indices $n_1, n_2, \ldots$ at all ε-orders.
- ◇ `o -> i`: face index i at ε-order o only.
- ◇ `{o1->i1, o2->i2, . . . }`: specific (ε-order, face) pairs.
- ◇ `Except[f]`: every face not matched by `f`.

Directories. Each `STIntegrate` call creates a working directory at

```
integrands/<ID>/
```

where `<ID>` is the problem identifier set by `"SetProblemID"`. Inside, the package creates one subdirectory per face integrand of the selected gauge, e.g.:

```
integrands/SubTropicaID$42_x3_1/ord_0_face_1/
```

Each face subdirectory contains:

- ◇ `counter_terms_integrands.m`: the list of counter-term integrands for that face
- ◇ `bestOrder.m`: the linearly reducible integration order found by `STFaster-Fubini` (see App. A.7)
- ◇ `partial_results/result_ct_<n>.m`: the result of each counter-term integration, written as it completes
- ◇ `result.m` and `successQ.m`: the combined face result and a boolean success flag, written once all counter-terms are done. The flag in `successQ.m` is read by `"ReuseExistingResults"` to decide whether a face can be skipped in a subsequent call

These files persist after the computation and can be read back at any time via `STReadResults[id]`. This is also how `STIntegrate` assembles the final answer:

it reads and sums all `result.m` files at the end of the run.

By default, the directory is wiped at the start of each call, so re-running the same diagram always starts from scratch. Setting

```
$STOverwritePreviousDirectories = False (* Default: True *);
```

before a computation preserves any existing directory from the time it is ran. Note that when working with `"StartAt"`, the preferred way to skip already-completed faces is `"ReuseExistingResults" -> True`, which operates face by face and does not require setting this global flag. The flag `$STOverwritePreviousDirectories = False` is more suited to manual workflows and debugging, where one wants to inspect or partially reuse a directory outside of the `"StartAt"` mechanism. ✧

A.3 Numerical evaluation: `STNIntegrate`

`STNIntegrate` numerically evaluates integrals. It accepts all the same input formats as `STIntegrate` (Forms 1–3), including generic parametric representations. The syntax is:

```
STNIntegrate[input,
"Method"              -> "pySecDec",
"Substitutions"       -> {},
"Order"               -> 0,
"MaxEval"             -> 10^7,
"Accuracy"            -> 10^-4,
"Integrator"          -> "Qmc",
"ContourDeformation" -> True,
"Verbose"             -> False,
"Dimension"           -> 4 - 2 eps,
"Normalization"       -> Automatic,
"Exponents"           -> Automatic,
"LoopMomenta"         -> Automatic,
"NumSamples"          -> 1,
"Seed"                -> Automatic,
"CacheDirectory"      -> Automatic]
```

Options. `"Method"` selects the backend: `"pySecDec"` (default), `"FIESTA"`, `"AMFlow"` or `"feyntrop"`. `"Substitutions"` specifies the numerical values for kinematic variables as a list of rules (e.g., `{s12 -> -7, mm -> 3}`). `"Order"` sets the ε-expansion depth. `"MaxEval"` caps the number of integrand evaluations. `"In-`

tegrator" selects the integration algorithm: "Qmc" (default, quasi-Monte Carlo), "Vegas", or "Disteval". "Qmc" is the default as it typically provides the fastest convergence. "LoopMomenta" follows the same convention as in STIntegrate. "NumSamples" -> n ($n > 1$) re-evaluates the integral at n independent kinematic points with prime-ratio components; the results are returned under the key "samples". "Seed" overrides the default graph-hash-based random seed. ✧

Return value. An Association with keys "Value" (the numerical result as a SeriesData), "Error" (integration uncertainties), "Method", "Integrator", "Substitutions", and "Timing" (wall-clock time in seconds). ✧

Numerical evaluation of hyperlogarithms: STToGinsh. STToGinsh[expr] numerically evaluates hyperlogarithm expressions via ginsh, the GiNaC interactive shell (requires ginsh installed and on the system path). It accepts Hlog, Mpl, mzv/zeta, and Log objects, converts them to GiNaC format internally, and returns a floating-point approximation. Expressions containing none of these are returned unchanged. A simple example of usage is:

```
(* Evaluate an expression with Hlogs and MZVs *)
result = STToGinsh[3 Hlog[2, {0, 1}] + mzv[3]]
(* Out: −6.20015 + 6.53276 I *)
```

A.4 Verification: STVerify

STVerify cross-checks a symbolic result from STIntegrate against a numerical evaluation. It automatically generates a Euclidean kinematic point, runs STNIntegrate for the numerical side and STToGinsh for the symbolic side, and compares Laurent coefficients order by order. The syntax is:

```
STVerify[input, result,
"Order"       -> 0,
"Dimension"   -> 4 - 2 eps,
"Tolerance"   -> 10^-3,
"MaxEval"     -> Automatic,
"Accuracy"    -> 10^-4,
"Integrator"      -> "Qmc",
"Method"          -> "pySecDec",
"Verbose"         -> False,
"NumSamples"      -> 1,
"Seed"            -> Automatic,
```

```
"Substitutions"  -> Automatic,
"Exponents"      -> Automatic]
```

Options. `"Tolerance"` (default 10^{-3}) sets the threshold for the relative error between numerical and symbolic coefficients. `"MaxEval" -> Automatic` defaults to 10^6 for one-loop integrals and 10^7 otherwise. `"NumSamples" -> n` ($n > 1$) re-verifies at n independent kinematic points and aggregates the results. `"Seed"` overrides the default hash-based random seed. `"Substitutions"` and `"Exponents"` are forwarded to `STNIntegrate` when present; `"Substitutions" -> Automatic` (default) auto-generates a Euclidean kinematic point. The remaining options are inherited from `STNIntegrate`. ✧

Return value. An `Association` with keys `"pass"` (`True`/`False`), `"reason"`, `"maxRelErr"`, `"relErr"` (per-coefficient relative errors), `"coefficients"` (detailed comparison at each ε-order), `"kinPoint"` (the kinematic point used), and `"symbolic"` (the evaluated symbolic result). ✧

A.5 Integration functions: `HyperIntica`

While `HyperIntica` is the main (analytic) integration routine working under the hood of the `STIntegrate` function, it can also be used as a standalone function. It evaluates iterated integrals $\int_0^\infty \cdots \int_0^\infty f \, \mathrm{d}x_1 \cdots \mathrm{d}x_n$ in (3.39) for expressions built from hyperlogarithms, polylogarithms, logarithms, and rational functions, returning results in terms of multi-zeta values and hyperlogarithms in the original integrand f parameters (if any). The syntax is:

```
HyperIntica[integrand, variables, "Monitor" -> False]
```

Required arguments. The argument `integrand` accepts either a symbolic expression involving `Hlog`, `Mpl`, `PolyLog`, logarithms and/or rational functions. The argument `variables` $= \{x_1, ..., x_n\}$ is the ordered list of integration variables, each x_i integrated, left to right, from 0 to ∞ by default. ✧

Optional arguments. Integration bounds other than $[0, \infty)$ can be specified in MATHEMATICA `Integrate`-style syntax: `HyperIntica[f, {x, 0, 1}, {y, 0, ∞}]`, where the allowed bounds are 0, 1, and ∞. The equivalent rule-based syntax `{x -> {0, 1}, y -> {0, ∞}}` is also accepted. The option `"Monitor" -> False` (default) prints progress to standard output as each variable is processed when set to `True`, useful for tracking, e.g., long multi-variable integrations. ✧

Return value. The function returns a combination of multi-zeta values, transcendental constants (such as log(2)), and hyperlogarithms in any remaining parameters. Some simple examples of usage are:

```
(* Univariate integral *)
HyperIntica[Log[z]/(1-z^2), {z}] (* Out: -(3 mzv[2])/2 *)

(* Two-variable integral with finite bounds *)
HyperIntica[1/((1+x)(1+y)), {x, 0, 1}, {y, 0, 1}] (* Out: Log[2]^2 *)

(* Monitor progress for a less trivial example *)
HyperIntica[Log[1+x/y]^2 Log[1+1/x] Log[y]/(x (1+y) (1+x+y)), {x, y}, "Monitor" -> True]
(* Out: (24 mzv[2]^2)/5 - 9 Log[2] mzv[2]^2 - 27/2 mzv[2] mzv[3] + (127 mzv[5])/8 *)
```

If the output of `HyperIntica` depends on `zeroInfPeriod`, one can try to simplify it in terms of multi-zeta values by applying `CleanZeroInf[]` to the output. ✧

Linear reducibility. The order of `variables` (i.e., the order in which we integrate) matters a lot: the algorithm requires each polynomial encountered during integration to factor linearly the current variable. When calling `HyperIntica` directly, a wrong order can silently return an incomplete (and thus incorrect) result. The two global flags `$NoAlgebraicRootsContributions` and `$HyperWarnZeroed` control the response to a non-linearly-factoring polynomial:

- ◇ `$NoAlgebraicRootsContributions = True` (default): the non-linear factor is set to zero. If `$HyperWarnZeroed = True` (default), two messages fire: `LinearFactors::nonlinear` identifies the offending polynomial and `LinearFactors::zeroed` reminds the user to verify the ordering. Setting `$HyperWarnZeroed = False` suppresses both reminders. In either case the result is 0, which is incorrect if the ordering is not linearly reducible.
- ◇ `$NoAlgebraicRootsContributions = False`: `HyperIntica` returns `$Failed` immediately upon encountering the non-linear factor, with message `LinearFactorsE::nonlinear`. This makes an incorrect ordering immediately visible.

Note that using `$NoAlgebraicRootsContributions = True` is safe when `HyperIntica` is called from `STIntegrate`, which verifies a valid order exists first, but it is in general advisable to verify a chosen order is linearly reducible by setting `$NoAlgebraicRootsContributions = False` before calling `HyperIntica` directly as a standalone function.

Both settings are illustrated by the following example:

```
(*Consider the following function to be integrated over x and y: *)
f = 1/(1 - x + x y^2);

(* Good order: 1-x+x*y^2 is linear in x; after integrating x the
   resulting letter y^2-1 =(y-1)(y+1) factors linearly in y *)
HyperIntica[f, {x, 0, 1}, {y, 0, 1}]   (* Out: 3 mzv[2]/2 *)

(* Bad order: x*y^2+(1-x) does not factor linearly in y *)

(* With $NoAlgebraicRootsContributions =True,
       $HyperWarnZeroed =True (defaults): *)
HyperIntica[f, {y, 0, 1}, {x, 0, 1}]
(* LinearFactors::nonlinear and LinearFactors::zeroed warnings fire
Out: 0 (incorrect!) *)

(* With $NoAlgebraicRootsContributions =False: *)
HyperIntica[f, {y, 0, 1}, {x, 0, 1}]
(* LinearFactorsE::nonlinear fires
Out: $Failed *)

(* Cross-check *)
Integrate[f, {x, 0, 1}, {y, 0, 1}]
(* Out: Pi^2/4 = 3 mzv[2]/2 *)
```

Divergence checking. The global flag `$HyperInticaCheckDivergences` controls whether `HyperIntica` computes and verifies boundary expansions at each integration step. In `HyperIntica.wl` the default is `True`: after finding the primitive in each variable, the code expands the result at $z = 0$ and $z = \infty$, collects all pole terms, and checks that they cancel. If a divergence is detected and `$HyperInticaAbortOnDivergence = True` (default), the computation aborts with `$Failed`. Setting `$HyperInticaAbortOnDivergence = False` instead stores the divergence (if any) in the association `$HyperInticaDivergences` and continues.

When the flag is set to `False`, the boundary expansions themselves are skipped entirely, not just the final cancellation check, so a speedup generally comes from avoiding the computation of the expansions altogether.

When `HyperIntica` is called from within `STIntegrate`, the flag is automatically set to `False`, since the tropical subtraction algorithm guarantees that every counter-term integrand passed to `HyperIntica` is locally finite. When using `Hy-`

perIntica as a standalone function on an integral whose finiteness is not known *a priori*, the user should keep $HyperInticaCheckDivergences = True and $HyperInticaAbortOnDivergence = True (the default in standalone mode). ✧

Memoization and cache management. During integration, HyperIntica accumulates several internal memoization caches. These persist for the duration of the session, which is beneficial when evaluating many related integrals, and can be cleared at any time by calling ForgetAllMemo in the notebook where calculations are done. ✧

Derivatives. HyperD[expr, var] computes the derivative of expr with respect to var, correctly implementing differentiation rules for Hlog and Mpl that the built-in MATHEMATICA D[] does not handle. expr may contain Hlog, Mpl, PolyLog, logarithms, and algebraic functions, while var can appear in both the upper limit and the letters of hyperlogarithms. Some examples are:

```
(* Differentiate Li2(1-x/u) with respect to x *)
HyperD[Mpl[{2}, {1 - x/u}], x]
(* Out: -Mpl[{1},{1-x/u}]/(u(1-x/u)) *)

(* Differentiate Hlog with var-dependent letters *)
HyperD[Hlog[z, {a, b}], a]
(* Out: (Hlog[z,{a}]-Hlog[z,{b}])/(a-b) - Hlog[z,{b}]/(z-a) *)

(* Product involving Mpl *)
HyperD[Log[x] Mpl[{1,2}, {x, y}], x]
(* Out: Log[x]/(x-1) (Mpl[{2},{x y}]/x - Mpl[{2},{y}]) + Mpl[{1,2},{x,y}]/x *)
```

Series. HyperSeries[expr, {var, point, order}] expands expr around var = point through $\mathcal{O}((\texttt{var} - \texttt{point})^{\texttt{order}})$, handling Hlog and Mpl when their arguments depend on the expansion variable (unlike the built-in Series). Some examples are:

```
(* Expand Li2(z) around z =0 *)
HyperSeries[Mpl[{2}, {z}], {z, 0, 5}]
(* Out: z+z^2/4+z^3/9+z^4/16+z^5/25 *)

(* Expand with parametric dependence *)
HyperSeries[Mpl[{2}, {1 - x/u}], {x, 0, 3}]
```

```
(* Out: Pi^2/6 + x(-1+Log[x/u])/u + x^2(-1+2 Log[x/u])/(4 u^2) + x^3(-1+3 Log[x/u])/(9 u^3) *)

(* Expand Hlog with argument approaching 0 *)
HyperSeries[Hlog[z, {0, 1}]/z, {z, 0, 4}]
(* Out: -1 - z/4 - z^2/9 - z^3/16 *)
```

Fibration basis. `FibrationBasis[expr, variables]` rewrites an expression in a fibration basis, often making non-trivial functional identities between hyperlogarithms manifest. The argument `expr` accepts a symbolic expression (e.g., `PolyLog` or `Hlog`). The `variables` argument defines the fibration order $x_1 \prec x_2 \prec \cdots \prec x_n$, where $w_j = w_j(x_1, \ldots, x_n)$. The output can vary significantly with this ordering. The decomposition is based on the shuffle identity introduced earlier in (3.43), which allows one to iteratively fiber off letters depending on x_k by evaluating them at the leading letter w_{j_1},

$$\mathrm{H}(x_k;\, w_{j_1}, w_{j_2}, \ldots, w_{j_w}) = \sum_{\vec{u} \sqcup\!\sqcup \vec{v} = (w_{j_2}, \ldots, w_{j_w})} \mathrm{H}(x_k;\, w_{j_1},\, \vec{u}\,) \cdot \mathrm{H}(w_{j_1};\, \vec{v}\,)\,,$$

where $\vec{u}$ contains only letters at levels $\prec x_k$ and $\mathrm{H}(w_{j_1}; \vec{v})$ provides the coefficient in the lower-level variables. Applying this recursively expresses any H as a polynomial in a basis of H's, where each word is lexicographically smaller than all of its proper suffixes. For example:

```
(* Single-variable: Li_5(-1/z) expressed as sum of c_w * Hlog[z, w] *)
FibrationBasis[PolyLog[5, -1/z], {z}]
(* Out: -Hlog[z, {0, 0, 0, 0, -1}] + Hlog[z, {0, 0, 0, 0, 0}]
        + Hlog[z, {0, 0, 0}] mzv[2] + (7 Hlog[z, {0}] mzv[2]^2)/10 *)

(* Two-variable: *)
FibrationBasis[Hlog[1, {1 + x, 1 + y}], {x, y}]
(* Out: -Hlog[x, {y, -1}] + Hlog[x, {y, 0}] - Hlog[x, {0}] Hlog[y, {-1}]
+ Hlog[x, {y}] Hlog[y, {-1}] + Hlog[x, {0}] Hlog[y, {0}]
- Hlog[x, {y}] Hlog[y, {0}] + Hlog[y, {0, -1}] - Hlog[y, {0, 0}] - mzv[2] *)
```

Symbol. `ConvertToSymbol[expr]` converts an expression containing hyperlogarithms and multiple polylogarithms to its "symbol" [92,93]. The available options are:

```
ConvertToSymbol[expr, "Expand" -> True,
                "DropConstants" -> False]
```

`"Expand" -> True` (default) factors each symbol letter into irreducible polynomials via `FactorList`, revealing the so-called "symbol alphabet." `"DropConstants" -> True` removes entries with only constant letters, following the same convention as `PolyLogTools` [41]. Some examples are:

```
(* Symbol of Li2(z) *)
ConvertToSymbol[PolyLog[2, z]]  (* Out: −Sym[{z, 1 − z}] *)

(* Symbol *)
ConvertToSymbol[Log[x] Log[1-x]]
(* Out: Sym[{1 − x, x}]+Sym[{x, 1 − x}] *)

(* Without factoring letters *)
ConvertToSymbol[Hlog[1, {0, 1-x}], "Expand" -> False]
(* Out: Sym[{1/(x−1), x/(1−x)}] *)
```

Notation conversions. `HyperIntica` uses two internal representations for iterated integrals: `Hlog[x, {letters}]` (hyperlogarithms with explicit upper limit) and `Mpl[{weights}, {args}]` (multiple polylogarithms in the $\mathrm{Li}_{n_1,\ldots}(z_1,\ldots)$ convention). The following replacement rules convert between them:

```
expr /. Hlog -> HlogAsMpl    (* Hlog → Mpl notation *)
expr /. Mpl -> MplAsHlog     (* Mpl → Hlog notation *)
```

These are used, for instance, when comparing results expressed in different bases or when interfacing with external tools that expect a specific notation.

When integrating over $[0,\infty)$, boundary contributions at 0 and ∞ may arise as formal `ZeroInfPeriod` objects. The replacement rule

```
expr /. ZeroInfPeriod -> ZeroInfPeriodAsMpl
```

expresses these boundary periods in terms of `Mpl` values (typically multiple zeta values), allowing the full result to be written in a uniform notation. ✧

A.6 Tropical subtraction utilities

This section documents the lower-level functions that implement the tropical subtraction pipeline. When using `STIntegrate`, these are called internally. They can also be invoked directly for non-standard integrals or debugging purposes (see Sec. 4 for a worked example).

Measure convention. The tropical utility functions (`STPreAnalysis`, `STTropicalContinuation`, `STExpandIntegral`) expect the integrand in the $\mathrm{d}\log\alpha_i$ convention (Sec. 1). When calling them directly on an integrand written in the $\mathrm{d}x_i$ convention, multiply by $\prod_i x_i$. The output of `STExpandIntegral` is returned in the $\mathrm{d}x_i$ convention, ready to be passed directly to `HyperIntica`. ✧

A.6.1 Tropical pre-analysis: `STPreAnalysis`

`STPreAnalysis` performs an extended analysis of the singularity structure of an Euler integrand. It is useful for diagnosing which divergences are and are not regulated by a given set of regulators.

```
STPreAnalysis[integrand, variables, coefficients
, True        (* facesQ, optional *)
, {eps}  ]    (* regulators, optional *)
```

Arguments. `integrand`, `variables`, and `coefficients` follow the same conventions as in `STExpandIntegral` below. The optional argument `facesQ` (default `True`) controls whether to compute the divergent faces. The optional `regulators` (default `{eps}`) lists the parameters used to determine which rays correspond to divergences. ✧

Return value. An `Association` with keys: `"trops"` (the value of the tropical integrand $\mathrm{Trop}\,\mathcal{I}(\rho)$ evaluated on each divergent ray ρ), `"rays"` (indices of the divergent rays), `"faces"` (the set Σ^{div} from (3.24)), `"us"` (the u-variables), and `"trData"` (the full tropical data structure including the rays, facets, vertices, and equations). A zero entry in `"trops"` indicates that the integrand has a logarithmic singularity along that ray which is *not* regulated by the specified regulators: the ε-dependent part of the tropical integrand vanishes on this ray, so ε alone cannot render the integral finite. Such rays typically signal the need of a secondary regulator (e.g., the parameter q in the small-x example of Sec. 4) and must be handled via `STTropicalContinuation` (App. A.6.2) before proceeding with the tropical subtraction. ✧

A.6.2 Nilsson–Passare continuation: `STTropicalContinuation`

`STTropicalContinuation` performs the Nilsson–Passare analytical continuation along a specified set of rays of the Newton polytope, extracting pole contributions before the subtraction formula is applied. It is called internally by `STExpandIn-`

tegral, but can also be invoked directly when finer control over the continuation is needed (see the small-x example in Sec. 4 for a worked illustration).

```
STTropicalContinuation[prefsInts, variables, rays]
```

Arguments. `prefsInts` is a list of pairs `{prefactor, integrand}`, where each pair represents a term in the Euler integral. `variables` is the list of integration variables. `rays` is a list of rays (integer vectors) along which the continuation is to be performed. Inputting an empty list `{}` returns the input unchanged. The function processes the rays recursively, one at a time. ✧

Return value. A list of pairs `{prefactor, integrand}` after the continuation has been performed. Each resulting term can then be passed to `STExpandIntegral` for tropical subtraction. ✧

A.6.3 Tropical subtraction: `STExpandIntegral`

`STExpandIntegral` is the core routine that reduces a (potentially divergent) Euler integral to a sum of locally finite integrals, applying the Nilsson–Passare procedure followed by tropical subtractions as described in Sec. 3.2.2. It takes three required and three optional positional arguments:

```
STExpandIntegral[integrand, variables, coeffs
, {eps}  (* regulators, optional *)
, {}     (* trDataGiven, optional *)
, {}   ] (* forceUs, optional *)
```

Required arguments. `integrand` is an Euler-type integrand, expressed as a product of polynomials raised to powers that depend on the dimensional regulator (see (1.1)). `variables` is the list of integration variables (Feynman or Schwinger parameters, denoted $x_j =$ `xj`), and `coeffs` lists the kinematic coefficients appearing in those polynomials. ✧

Return value. The function returns a list `{{prefactor, subtraction}, ...}`, one entry per Nilsson–Passare continuation term. Each `subtraction` is itself a list `{{faceIntegrand, faceVariables}, ...}` indexed by the divergent faces of the Newton polytope. Here `faceIntegrand` is a list of additive terms whose sum is locally finite and ready to be passed to `HyperIntica`. ✧

Optional arguments. `regulators` (default `{eps}`) lists the dimensional regulators used to identify which rays of the Newton polytope correspond to divergences. `trDataGiven` (default `{}`) accepts pre-computed tropical data, skipping the `PolyMake` call. When `STIntegrate` is used, tropical data for all gauge candidates is computed concurrently upfront and passed through this argument. `forceUs` (default `{}`) pins specific u-variables for both the NP continuation and the subtraction formula. ✧

A.6.4 Scaling analysis: `STFactor`

`STFactor` factorizes products inside powers that depend on ε: it splits $(a{\cdot}b)^{c\varepsilon+d}$ into $a^{c\varepsilon+d}\,b^{c\varepsilon+d}$ whenever one of the factors is manifestly positive for positive integration variables:

```
STFactor[expression]
```

A typical usage is to rescale the integration variables along a ray ρ of the Newton polytope (e.g., $x_i \to x_i/\lambda$ for each component $\rho_i > 0$) and then apply `STFactor` to the result. The factorized form makes the λ-dependence explicit, revealing whether the integrand diverges or converges along that ray. ✧

A.6.5 Series mapping: `STMapSeries`

`STMapSeries` applies a given function to each coefficient of a Mathematica `SeriesData` object and returns the result as a new `SeriesData`. For example, if the locally finite integrand has been expanded in ε via `Series`, one can integrate each order separately:

```
STMapSeries[function, series]
```

A typical usage is `STMapSeries[HyperIntica[#, {x1, x2}]&, series]`, which calls `HyperIntica` on the integrand at each order in ε and reassembles the results into a series. ✧

A.7 Linear reducibility: `STFasterFubini`

`STFasterFubini` finds a linearly reducible integration order for a given set of polynomials, using an optimized implementation of the Fubini reduction algorithm discussed in Sec. 3.4. It is called internally by `STIntegrate` to determine valid

integration orders before dispatching to `HyperIntica`.[9] The syntax is:

```
STFasterFubini[groupPolynomials, variables,
"SolverBound" -> 10^9,
"FindRoots"   -> False,
"Heuristic"   -> "LeafCountLinear"]
```

Arguments. The argument `groupPolynomials` is a list of polynomial lists (see, e.g., Sec. 4.4), one per counter-term integrand; the algorithm finds an order that is linearly reducible simultaneously for all groups. The `variables` is the list of integration variables. The function returns a permutation of `variables` giving a valid integration order, or `NOLR` if none is found. ✧

Options. `SolverBound` (default 10^9) caps the number of terms in polynomials during discriminant and resultant computations; polynomials above this bound are skipped, which can speed up the search at the cost of potentially missing some valid orders. `FindRoots -> True` attempts to factor univariate polynomials of degree greater than one by introducing auxiliary root variables, extending the class of linearly reducible integrals at the cost of increased complexity (default `False`). `"Heuristic"` (default `"LeafCountLinear"`) controls how candidate orders are scored. See the description under `STIntegrate` (see Par. A.2) for the full list of available heuristics. ✧

B Notation used in `HyperIntica`

This appendix collects the conventions used by `HyperIntica`, following those of Panzer's `HyperInt` [5]. The material overlaps with the review in Sec. 3.3 but is presented here in self-contained form for reference.

Hyperlogarithms and words. As introduced in Sec. 3.3, the hyperlogarithm $\mathrm{H}(z; w_1, \ldots, w_n)$ is the iterated integral

$$\mathrm{H}(z; w_1, \ldots, w_n) = \int_{0<t_n<\cdots<t_1<z} \frac{\mathrm{d}t_1}{t_1 - w_1} \frac{\mathrm{d}t_2}{t_2 - w_2} \cdots \frac{\mathrm{d}t_n}{t_n - w_n}\,, \tag{B.1}$$

[9] Users running Mathematica < 14.x may encounter a kernel crash during the linear-order finding step, caused by a bug in the Flint library (`fmpz_mpoly_pfrac_init: internal error`) shipped with that version. The bug affects `FactorList` on certain multivariate polynomials arising in complicated examples inside the internal function `STFubini`, and is fixed in Mathematica 14.x.

or equivalently $\mathrm{H}(z; w_1, \ldots, w_n) = \int_0^z \frac{\mathrm{d}t}{t-w_1} \mathrm{H}(t; w_2, \ldots, w_n)$ with $\mathrm{H}(z; \emptyset) = 1$. The sequence $w = [w_1, \ldots, w_n]$ is called a *word*, its entries are the *letters*, and the integer $n = |w|$ is the *weight*. For example, the relevant alphabets are $\Sigma = \{0, 1\}$ for classical multi-zeta values and $\Sigma = \{-1, 0, 1\}$ for alternating Euler sums.

Internally, `HyperIntica` stores linear combinations of hyperlogarithms as *wordlists*: lists of (coefficient, word) pairs

$$\texttt{wordlist} = \big\{\{c_1, w_1\}, \{c_2, w_2\}, \ldots\big\} \quad \longleftrightarrow \quad \sum_i c_i \cdot \mathrm{H}(z; w_i)\,. \tag{B.2}$$

Shuffle product. The shuffle product $u \sqcup\!\sqcup v$ of two words $u = [u_1, \ldots, u_m]$ and $v = [v_1, \ldots, v_n]$ is the sum over all interleavings that preserve the relative order within each word (cf. (3.43)):

$$u \sqcup\!\sqcup v = \sum_{\sigma \in \Sigma(m,n)} \sigma(u, v)\,, \tag{B.3}$$

where $\Sigma(m, n)$ is the set of (m, n)-shuffles. It satisfies the recursion

$$[a, u'] \sqcup\!\sqcup [b, v'] = \big[a, (u' \sqcup\!\sqcup [b, v'])\big] + \big[b, ([a, u'] \sqcup\!\sqcup v')\big]\,, \tag{B.4}$$

with $u \sqcup\!\sqcup \emptyset = \emptyset \sqcup\!\sqcup u = u$, and the key algebraic property is that hyperlogarithms satisfy a shuffle algebra:

$$\mathrm{H}(z; u) \cdot \mathrm{H}(z; v) = \sum_{w \in u \sqcup\!\sqcup v} \mathrm{H}(z; w)\,. \tag{B.5}$$

Shuffle regularization. Iterated integrals diverge when letters coincide with integration boundaries. Shuffle regularization exploits the algebra (B.5) to systematically extract finite values. The basic idea is that the shuffle product lets us isolate divergent contributions; for instance, for a word $[a, 0]$ with one trailing zero,

$$\mathrm{H}(z; a, 0) = \mathrm{H}(z; a) \cdot \log(z) - \mathrm{H}(z; 0, a)\,, \tag{B.6}$$

where $\mathrm{H}(z; 0, a)$ is convergent as $z \to 0$ and the divergence is carried entirely by $\log(z)$. Regularization then discards the pure-logarithm divergences while preserving all algebraic relations.

As discussed in the main text, `HyperIntica` uses three regularizations, all following this pattern. They are summarized in the table below:

Reg.	Boundary	Divergent word structure	Divergence
Reg_0	$z \to 0$	$[w_1, \ldots, w_{n-k}, \underbrace{0, \ldots, 0}_{k}]$	$\frac{\log(z)^k}{k!}$
Reg_σ	$z \to \sigma$	$[\underbrace{\sigma, \ldots, \sigma}_{k}, w_{k+1}, \ldots, w_n]$	$\frac{(-\log(\sigma - z))^k}{k!}$
Reg_∞	$z \to \infty$	$[w_1, \ldots, w_{n-k}, \underbrace{-1, \ldots, -1}_{k}]$	$\frac{\log(z)^k}{k!}$

Explicitly, for a word with k divergent letters at the relevant boundary, the regularization shuffles with the pure-divergent word and applies an alternating sign:

$$\begin{aligned}
\mathrm{Reg}_0\mathrm{H}(z; [w_1, \ldots, w_{n-k}, 0^k]) &\equiv \sum_{u \in [w_1, \ldots, w_{n-k-1}] \sqcup\!\sqcup [0]^k} (-1)^k \mathrm{H}(z; [u, w_{n-k}])\,, \\
\mathrm{Reg}_\sigma\mathrm{H}(\sigma; [\sigma^k, w_{k+1}, \ldots, w_n]) &\equiv \sum_{u \in [\sigma]^k \sqcup\!\sqcup [w_{k+2}, \ldots, w_n]} (-1)^k \mathrm{H}(\sigma; [w_{k+1}, u])\,, \\
\mathrm{Reg}_\infty\mathrm{H}(\infty; [w_1, \ldots, w_{n-k}, (-1)^k]) &\equiv \sum_{u \in [w_1, \ldots, w_{n-k-1}] \sqcup\!\sqcup [-1]^k} (-1)^k \mathrm{H}(\infty; [u, w_{n-k}])\,.
\end{aligned}$$

Words consisting entirely of the divergent letter regularize to zero. Note the asymmetry: Reg_σ acts on *leading* entries (closest to the upper bound z) while Reg_0 acts on *trailing* entries (closest to the lower bound 0), reflecting the ordering $0 < t_n < \cdots < t_1 < z$ in the iterated integral. For illustration,

$$\begin{aligned}
\mathrm{Reg}_0\mathrm{H}(z; [a, 0]) &= -\mathrm{H}(z; [0, a])\,, \\
\mathrm{Reg}_1\mathrm{H}(1; [1, a]) &= -\mathrm{H}(1; [a, 1])\,, \\
\mathrm{Reg}_1\mathrm{H}(1; [1, a, b]) &= -\mathrm{H}(1; [a, 1, b]) - \mathrm{H}(1; [a, b, 1])\,, \\
\mathrm{Reg}_\infty\mathrm{H}(\infty; [-1]) &= 0\,.
\end{aligned}$$

Periods. The fully regularized periods combine both boundary regularizations:

$$\texttt{zeroOnePeriod}(w) = \mathrm{Reg}_1\mathrm{Reg}_0\mathrm{H}(1; w)\,, \tag{B.7a}$$

$$\texttt{zeroInfPeriod}(w) = \mathrm{Reg}_\infty\mathrm{Reg}_0\mathrm{H}(\infty; w)\,. \tag{B.7b}$$

After regularization, the results are finite linear combinations of convergent hyperlogarithms, expressible in terms of multi-zeta values. A crucial property is that

shuffle regularization respects the shuffle product [76],

$$\mathrm{Reg}\big(\mathrm{H}(z;u)\cdot\mathrm{H}(z;v)\big) = \mathrm{Reg}\left(\sum_{w\in u\sqcup\!\sqcup v}\mathrm{H}(z;w)\right), \tag{B.8}$$

ensuring consistency of the integration algorithm.

Multi-zeta values and classical functions as hyperlogarithms. The multiple polylogarithm (MPL) with positive integer indices $n_1,\ldots,n_k$ and arguments $z_1,\ldots,z_k\in\mathbb{C}$ is

$$\mathrm{Li}_{n_1,\ldots,n_k}(z_1,\ldots,z_k) = \sum_{m_1>m_2>\cdots>m_k\geqslant 1}\frac{z_1^{m_1}z_2^{m_2}\cdots z_k^{m_k}}{m_1^{n_1}m_2^{n_2}\cdots m_k^{n_k}}, \tag{B.9}$$

converging for $|z_1|\leqslant 1$ (with $|z_1|<1$ or $n_1\geqslant 2$ at the boundary). For nonzero integers $n_1,\ldots,n_k$ with $|n_1|\geqslant 2$, the multiple zeta value (MZV), also called alternating Euler sum when negative indices appear, is the specialization

$$\begin{aligned}\zeta(n_1,n_2,\ldots,n_k) &\equiv \mathrm{Li}_{|n_1|,\ldots,|n_k|}\left(\frac{n_1}{|n_1|},\ldots,\frac{n_k}{|n_k|}\right)\\ &= \sum_{m_1>\cdots>m_k\geqslant 1}\frac{(\mathrm{sgn}n_1)^{m_1}\cdots(\mathrm{sgn}n_k)^{m_k}}{m_1^{|n_1|}\cdots m_k^{|n_k|}},\end{aligned} \tag{B.10}$$

where $\mathrm{sgn}n_i = n_i/|n_i|\in\{-1,+1\}$. When all $n_i>0$, this is a classical MZV. When at least one index is negative, the alternating signs $(-1)^{m_i}$ give an alternating Euler sum. The weight is $|n_1|+\cdots+|n_k|$ and the depth is k. Some familiar special cases are

$$\begin{gathered}\zeta(2)=\pi^2/6,\quad \zeta(-2)=-\pi^2/12,\quad \zeta(-1)=-\log 2\,,\\ \zeta(1,2)=\zeta(3),\quad \zeta(-1,2)=-\tfrac{3}{2}\log(2)\,\zeta(2)+\zeta(3)\,.\end{gathered}$$

In `HyperIntica`, these can be generated using the rule

```
mzv[1, 2, 1, 2, 1] //. mzvAllReductions
(* Out:  -(1/30) Pi^4 Zeta[3] + 3 Zeta[7] *)
```

up to weight seven.

The connection to hyperlogarithms is that, for positive indices $n_1,\ldots,n_k$ with

$n_1 \geqslant 2$ [76],

$$\zeta(n_1, \ldots, n_k) = (-1)^k \mathrm{H}\Big(1; \underbrace{0, \ldots, 0}_{n_k - 1}, 1, \ldots, \underbrace{0, \ldots, 0}_{n_1 - 1}, 1\Big) . \tag{B.11}$$

More generally, for $n_i \in \mathbb{Z} \setminus \{0\}$ the word uses letters in $\{-1, 0, 1\}$, with each index n_i contributing $|n_i| - 1$ zeros followed by a letter encoding the sign transitions.

For reference, we collect the standard dictionary. The natural logarithm is $\log(z) = \mathrm{Reg}_0 \mathrm{H}(z; 0)$, while the classical polylogarithm $\mathrm{Li}_n(z) = \sum_{k=1}^{\infty} z^k / k^n$ reads

$$\mathrm{Li}_n(z) = -\mathrm{H}\big(z; \underbrace{0, \ldots, 0}_{n-1}, 1\big) , \tag{B.12}$$

giving in particular $\mathrm{Li}_1(z) = -\log(1 - z) = -\mathrm{H}(z; 1)$, $\mathrm{Li}_2(z) = -\mathrm{H}(z; 0, 1)$, and $\mathrm{Li}_3(z) = -\mathrm{H}(z; 0, 0, 1)$. More generally, the MPL is expressed as

$$\mathrm{Li}_{n_1, \ldots, n_k}(z_1, \ldots, z_k) = (-1)^k \mathrm{H}\Big(1; \underbrace{0, \ldots, 0}_{n_k - 1}, \frac{1}{\prod_{j=k}^{k} z_j}, \ldots, \underbrace{0, \ldots, 0}_{n_1 - 1}, \frac{1}{\prod_{j=1}^{k} z_j}\Big) .$$

References

[1] G. Salvatori, *The tropical geometry of subtraction schemes*, 2406.14606.

[2] A. Hillman, *A subtraction scheme for feynman integrals*, 2311.03439.

[3] F. Brown and C. Dupont, *Single-valued integration and superstring amplitudes in genus zero*, *Commun. Math. Phys.* **382** (2021) 815 [1910.01107].

[4] F. Brown, *The massless higher-loop two-point function*, *Commun. Math. Phys.* **287** (2009) 925 [0804.1660].

[5] E. Panzer, *Algorithms for the symbolic integration of hyperlogarithms with applications to feynman integrals*, *Computer Physics Communications* **188** (2015) 148–166.

[6] O. Schnetz, *Numbers and Functions in Quantum Field Theory*, *Phys. Rev. D* **97** (2018) 085018 [1606.08598].

[7] E. Panzer, *Algorithms for the symbolic integration of hyperlogarithms with applications to Feynman integrals*, *Comput. Phys. Commun.* **188** (2015) 148 [1403.3385].

[8] A. Kardos, S.-O. Moch and O. Schnetz, *HyperFORM – a FORM package for parametric integration with hyperlogarithms*, 2511.19992.

[9] B. Ruijl, T. Ueda and J.A.M. Vermaseren, *Forcer, a FORM program for the parametric reduction of four-loop massless propagator diagrams*, *Comput. Phys. Commun.* **253** (2020) 107198 [1704.06650].

[10] T. Hahn, *Generating Feynman diagrams and amplitudes with FeynArts 3*, *Comput. Phys. Commun.* **140** (2001) 418 [hep-ph/0012260].

[11] T. Hahn and M. Perez-Victoria, *Automatized one loop calculations in four-dimensions and D-dimensions*, *Comput. Phys. Commun.* **118** (1999) 153 [hep-ph/9807565].

[12] V. Shtabovenko, R. Mertig and F. Orellana, *FeynCalc 10: Do multiloop integrals dream of computer codes?*, *Comput. Phys. Commun.* **306** (2025) 109357 [2312.14089].

[13] H.H. Patel, *Package-X: A Mathematica package for the analytic calculation of one-loop integrals*, *Comput. Phys. Commun.* **197** (2015) 276 [1503.01469].

[14] H.H. Patel, *Package-X 2.0: A Mathematica package for the analytic calculation of one-loop integrals*, *Comput. Phys. Commun.* **218** (2017) 66 [1612.00009].

[15] V. Shtabovenko, *FeynHelpers: Connecting FeynCalc to FIRE and Package-X*, *Comput. Phys. Commun.* **218** (2017) 48 [1611.06793].

[16] A.V. Smirnov and M. Zeng, *FIRE 6.5: Feynman integral reduction with new simplification library*, *Comput. Phys. Commun.* **302** (2024) 109261 [2311.02370].

[17] A. von Manteuffel and C. Studerus, *Reduze 2 - Distributed Feynman Integral Reduction*, 1201.4330.

[18] R.N. Lee, *LiteRed 1.4: a powerful tool for reduction of multiloop integrals*, *J. Phys. Conf. Ser.* **523** (2014) 012059 [1310.1145].

[19] P. Maierhöfer, J. Usovitsch and P. Uwer, *Kira—A Feynman integral reduction program*, *Comput. Phys. Commun.* **230** (2018) 99 [1705.05610].

[20] J. Klappert, F. Lange, P. Maierhöfer and J. Usovitsch, *Integral reduction with Kira 2.0 and finite field methods*, *Comput. Phys. Commun.* **266** (2021) 108024 [2008.06494].

[21] A. Georgoudis, K.J. Larsen and Y. Zhang, *Azurite: An algebraic geometry based package for finding bases of loop integrals*, *Comput. Phys. Commun.* **221** (2017) 203 [1612.04252].

[22] Z. Wu, J. Boehm, R. Ma, H. Xu and Y. Zhang, *NeatIBP 1.0, a package generating small-size integration-by-parts relations for Feynman integrals*, *Comput. Phys. Commun.* **295** (2024) 108999 [2305.08783].

[23] X. Guan, X. Liu, Y.-Q. Ma and W.-H. Wu, *Blade: A package for block-triangular form improved Feynman integrals decomposition*, *Comput. Phys. Commun.* **310** (2025) 109538 [2405.14621].

[24] T. Peraro, *FiniteFlow: multivariate functional reconstruction using finite fields and dataflow graphs*, *JHEP* **07** (2019) 031 [1905.08019].

[25] C. Meyer, *Algorithmic transformation of multi-loop master integrals to a canonical basis with CANONICA*, *Comput. Phys. Commun.* **222** (2018) 295 [1705.06252].

[26] R.N. Lee, *Libra: A package for transformation of differential systems for multiloop integrals*, *Comput. Phys. Commun.* **267** (2021) 108058 [2012.00279].

[27] M. Hidding, *DiffExp, a Mathematica package for computing Feynman integrals in terms of one-dimensional series expansions*, *Comput. Phys. Commun.* **269** (2021) 108125 [2006.05510].

[28] O. Gituliar and V. Magerya, *Fuchsia: a tool for reducing differential equations for Feynman master integrals to epsilon form*, *Comput. Phys. Commun.* **219** (2017) 329 [1701.04269].

[29] M. Prausa, *epsilon: A tool to find a canonical basis of master integrals*, *Comput. Phys. Commun.* **219** (2017) 361 [1701.00725].

[30] C. Dlapa, J. Henn and K. Yan, *Deriving canonical differential equations for Feynman integrals from a single uniform weight integral*, *JHEP* **05** (2020) 025 [2002.02340].

[31] G. Heinrich, S.P. Jones, M. Kerner, V. Magerya, A. Olsson and J. Schlenk, *Numerical scattering amplitudes with pySecDec*, *Comput. Phys. Commun.* **295** (2024) 108956 [2305.19768].

[32] A.V. Smirnov, N.D. Shapurov and L.I. Vysotsky, *Fiesta5: Numerical high-performance feynman integral evaluation*, *Comput. Phys. Commun.* **277** (2022) 108386 [2110.11660].

[33] X. Liu and Y.-Q. Ma, *AMFlow: A Mathematica package for Feynman integrals computation via auxiliary mass flow*, *Comput. Phys. Commun.* **283** (2023) 108565 [2201.11669].

[34] M. Borinsky, H.J. Munch and F. Tellander, *Tropical Feynman integration in the Minkowski regime*, *Comput. Phys. Commun.* **292** (2023) 108874 [2302.08955].

[35] D. Maitre, *HPL, a mathematica implementation of the harmonic polylogarithms*, *Comput. Phys. Commun.* **174** (2006) 222 [hep-ph/0507152].

[36] T. Huber and D. Maître, *HypExp, a Mathematica package for expanding hypergeometric functions around integer-valued parameters*, *Comput. Phys. Commun.* **175** (2006) 122 [hep-ph/0507094].

[37] T. Huber and D. Maître, *HypExp 2, Expanding hypergeometric functions about half-integer parameters*, *Comput. Phys. Commun.* **178** (2008) 755 [0708.2443].

[38] C. Fevola, S. Mizera and S. Telen, *Landau Singularities Revisited: Computational Algebraic Geometry for Feynman Integrals*, *Phys. Rev. Lett.* **132** (2024) 101601 [2311.14669].

[39] C. Fevola, S. Mizera and S. Telen, *Principal Landau determinants*, *Comput. Phys. Commun.* **303** (2024) 109278 [2311.16219].

[40] M. Correia, M. Giroux and S. Mizera, *SOFIA: Singularities of Feynman integrals automatized*, *Comput. Phys. Commun.* **320** (2026) 109970 [2503.16601].

[41] C. Duhr and F. Dulat, *PolyLogTools — polylogs for the masses*, *JHEP* **08** (2019) 135 [1904.07279].

[42] X. Jiang, J. Liu, X. Xu and L.L. Yang, *Symbol letters of Feynman integrals from Gram determinants*, *Phys. Lett. B* **864** (2025) 139443 [2401.07632].

[43] B. Ananthanarayan, S. Banik, S. Bera and S. Datta, *FeynGKZ: A Mathematica package for solving Feynman integrals using GKZ hypergeometric systems*, *Comput. Phys. Commun.* **287** (2023) 108699 [2211.01285].

[44] J.L. Bourjaily, *Efficient Tree-Amplitudes in N=4: Automatic BCFW Recursion in Mathematica*, 1011.2447.

[45] D. Maitre and P. Mastrolia, *S@M, a Mathematica Implementation of the Spinor-Helicity Formalism*, *Comput. Phys. Commun.* **179** (2008) 501 [0710.5559].

[46] M. Accettulli Huber, *SpinorHelicity4D: a Mathematica toolbox for the four-dimensional spinor-helicity formalism*, 2304.01589.

[47] J.L. Bourjaily, *Computational Tools for Trees in Gauge Theory and Gravity*, 2312.17745.

[48] J. Goode, F. Herzog and S. Teale, *OPITeR: A program for tensor reduction of multi-loop Feynman integrals*, *Comput. Phys. Commun.* **312** (2025) 109606 [2411.02233].

[49] H.S. Hannesdottir and S. Mizera, *What is the i epsilon for the S-matrix?*, Springer International Publishing (2022), 10.1007/978-3-031-18258-7.

[50] SubTropica GitHub repository, 2026.

[51] E. Gawrilow and M. Joswig, `polymake`*: a framework for analyzing convex polytopes*, in *Polytopes—combinatorics and computation (Oberwolfach, 1997)*, vol. 29 of *DMV Sem.*, pp. 43–73, Birkhäuser, Basel (2000).

[52] B. Assarf, E. Gawrilow, K. Herr, M. Joswig, B. Lorenz, A. Paffenholz et al., *Computing convex hulls and counting integer points with* `polymake`, *Math. Program. Comput.* **9** (2017) 1.

[53] C.W. Bauer, A. Frink and R. Kreckel, *Introduction to the GiNaC framework for symbolic computation within the C++ programming language*, *J. Symb. Comput.* **33** (2002) 1 [cs/0004015].

[54] V. Chestnov and G. Crisanti, *Sampling Polynomial Rational Remainders with SPQR: A new Package for Polynomial Division and Elimination*, 2511.14875.

[55] M. Howell et al., *Homebrew: The missing package manager for macOS (or Linux)*, 2009.

[56] M. Borinsky, H.J. Munch and F. Tellander, *Tropical feynman integration in the minkowski regime*, *Computer Physics Communications* **292** (2023) 108874.

[57] E. Panzer, *On hyperlogarithms and feynman integrals with divergences and many scales*, *Journal of High Energy Physics* **2014** (2014) .

[58] L. Nilsson and M. Passare, *Mellin transforms of multivariate rational functions*, *Journal of Geometric Analysis* **23** (2013) 24.

[59] M. Borinsky, *Tropical monte carlo quadrature for feynman integrals*, *Ann. Inst. H. Poincare D Comb. Phys. Interact.* **10** (2023) 635 [2008.12310].

[60] B. Jantzen, A.V. Smirnov and V.A. Smirnov, *Expansion by regions: revealing potential and glauber regions automatically*, *Eur. Phys. J. C* **72** (2012) 2139 [1206.0546].

[61] D.R. Grayson and M.E. Stillman, "Macaulay2, a software system for research in algebraic geometry." Available at http://www2.macaulay2.com.

[62] D.J. Bates, J.D. Hauenstein, A.J. Sommese and C.W. Wampler, "Bertini: Software for numerical algebraic geometry." Available at http://bertini.nd.edu with permanent doi: dx.doi.org/10.7274/R0H41PB5.

[63] H.S. Hannesdottir and S. Mizera, *What is the $i\varepsilon$ for the S-matrix?*, SpringerBriefs in Physics, Springer (1, 2023), 10.1007/978-3-031-18258-7, [2204.02988].

[64] E. Gardi, F. Herzog, S. Jones and Y. Ma, *Dissecting polytopes: Landau singularities and asymptotic expansions in $2 \to 2$ scattering*, *JHEP* **08** (2024) 127 [2407.13738].

[65] N. Arkani-Hamed, A. Hillman and S. Mizera, *Feynman polytopes and the tropical geometry of uv and ir divergences*, *Phys. Rev. D* **105** (2022) 125013 [2202.12296].

[66] G.M. Ziegler, *Lectures on Polytopes*, vol. 152 of *Graduate Texts in Mathematics*, Springer-Verlag, New York (1995).

[67] C. Berkesch, J. Forsgård and M. Passare, *Euler-mellin integrals and a-hypergeometric functions*, *Michigan Mathematical Journal* **63** (2014) 101.

[68] K. Schultka, *Toric geometry and regularization of feynman integrals*, 1806.01086.

[69] D. Cox, "What is a toric variety?."

[70] R.P. Stanley, *Enumerative Combinatorics: Volume 1*, Cambridge University Press (2011), 10.1017/CBO9781139058520.

[71] F.C.S. Brown, *On the periods of some feynman integrals*, 2010.

[72] F. Chavez and C. Duhr, *Three-mass triangle integrals and single-valued polylogarithms*, *JHEP* **11** (2012) 114 [1209.2722].

[73] J. Ablinger, J. Blumlein, A. Hasselhuhn, S. Klein, C. Schneider and F. Wissbrock, *Massive 3-loop Ladder Diagrams for Quarkonic Local Operator Matrix Elements*, *Nucl. Phys. B* **864** (2012) 52 [1206.2252].

[74] C. Anastasiou, C. Duhr, F. Dulat and B. Mistlberger, *Soft triple-real radiation for Higgs production at N3LO*, *JHEP* **07** (2013) 003 [1302.4379].

[75] C. Anastasiou, C. Duhr, F. Dulat, F. Herzog and B. Mistlberger, *Real-virtual contributions to the inclusive Higgs cross-section at N^3LO*, *JHEP* **12** (2013) 088 [1311.1425].

[76] E. Panzer, *Feynman integrals and hyperlogarithms*, Ph.D. thesis, Humboldt U., 2015. 1506.07243. 10.18452/17157.

[77] S. Weinzierl, *Feynman Integrals. A Comprehensive Treatment for Students and Researchers*, UNITEXT for Physics, Springer (2022), 10.1007/978-3-030-99558-4, [2201.03593].

[78] O. Barrera, A. Dersy, R. Husain, M.D. Schwartz and X. Zhang, *Analytic regression of Feynman integrals from high-precision numerical sampling*, *JHEP* **01** (2026) 014 [2507.17815].

[79] E. Gardi and Z. Zhu, *Computing the soft anomalous dimension with massless particles using the method of regions*, *JHEP* **03** (2026) 173 [2509.18017].

[80] D. Chicherin, G.P. Korchemsky, E. Sokatchev and A. Zhiboedov, *Energy correlators in four-dimensional gravity*, 2512.23791.

[81] G. Brunello, S. Caron-Huot, G. Crisanti, M. Giroux and S. Smith, *High-energy evolution in planar QCD to three loops: the non-conformal contribution*, *JHEP* **11** (2025) 055 [2508.03794].

[82] I. Balitsky and G.A. Chirilli, *Next-to-leading order evolution of color dipoles*, *Phys. Rev. D* **77** (2008) 014019 [0710.4330].

[83] F. Herzog, *Geometric IR subtraction for final state real radiation*, *JHEP* **08** (2018) 006 [1804.07949].

[84] V.A. Smirnov and F. Wunder, *Expansion by regions meets angular integrals*, *JHEP* **08** (2024) 138 [2405.13120].

[85] M. Simonović, T. Baldauf, M. Zaldarriaga, J.J. Carrasco and J.A. Kollmeier, *Cosmological perturbation theory using the FFTLog: formalism and connection to QFT loop integrals*, *JCAP* **04** (2018) 030 [1708.08130].

[86] A. Herderschee, *Feynman-like parameterizations of (anti-)de Sitter Witten diagrams for all masses at any loop order*, 2509.02699.

[87] S. Caron-Huot and M. Herranen, *High-energy evolution to three loops*, *JHEP* **02** (2018) 058 [1604.07417].

[88] C. Bogner and S. Weinzierl, *Resolution of singularities for multi-loop integrals*, *Comput. Phys. Commun.* **178** (2008) 596 [0709.4092].

[89] J. Henn, B. Mistlberger, V.A. Smirnov and P. Wasser, *Constructing d-log integrands and computing master integrals for three-loop four-particle scattering*, *JHEP* **04** (2020) 167 [2002.09492].

[90] P. Bargiela, F. Caola, A. von Manteuffel and L. Tancredi, *Three-loop helicity amplitudes for diphoton production in gluon fusion*, *JHEP* **02** (2022) 153 [2111.13595].

[91] R.N. Lee and A.A. Pomeransky, *Critical points and number of master integrals*, *JHEP* **11** (2013) 165 [1308.6676].

[92] A.B. Goncharov, M. Spradlin, C. Vergu and A. Volovich, *Classical Polylogarithms for Amplitudes and Wilson Loops*, *Phys. Rev. Lett.* **105** (2010) 151605 [1006.5703].

[93] C. Duhr, H. Gangl and J.R. Rhodes, *From polygons and symbols to polylogarithmic functions*, *JHEP* **10** (2012) 075 [1110.0458].